Shortest paths

Outline

- Shortest path problems
  - Negative cycles
  - Optimality of subpaths
- Optimality conditions and potentials
- The linear programming formulation
  - The dual model and the potentials
  - The single-source model
- Shortest paths in acyclic graphs
- The Bellman–Ford algorithm
  - Bellman–Ford on the reference graph
- Dijkstra's algorithm
  - Dijkstra's algorithm on the reference graph
  - Negative lengths
- Typical mistakes
- In summary

Learning objectives

At the end of this chapter you will be able to:

- state the s–t and the single-source shortest path problems, and recognize when a negative cycle makes them ill-posed;
- use the optimality of subpaths, Bellman's equations and the shortest path tree;
- write the LP model of a shortest path as a unit flow, write its dual with the potentials π_i, and use feasible potentials as a certificate of optimality;
- run the algorithm for acyclic graphs, the Bellman–Ford algorithm (with the detection of a negative cycle) and Dijkstra's algorithm, prove their correctness and give their running time.

Prerequisites

- From the chapter «Graphs»: directed walks, paths and cycles, the node–arc incidence matrix, trees and the reference graph.
- From the module Linear Programming, the chapter «Duality theory»: the dual of an LP model and the weak duality theorem.

Shortest path problems

Introduction: Shortest path problems

- Every arc of a directed graph gets a **length**, and we look for the directed paths of minimum length from a source node
- Shortest paths are everywhere: road navigation, routing in communication networks, and as a building block of many algorithms on flows later in the course
- The lengths may be negative: we will see when this makes the problem ill-posed (negative cycles) and which properties the distances satisfy when it does not

Definition 1: shortest path problems

Let $G = (N, A)$ be a directed graph with a ***length*** *$d_{ij} \in \mathbb{R}$ for every arc $(i, j) \in A$, of any sign.*

- *The length of a directed walk $W = i_0, i_1, \ldots, i_k$ is the sum of the lengths of its arcs, $d(W) = \sum_{t=1}^{k} d_{i_{t-1} i_t}$.*
- *Given two nodes s and t, the s–t* ***shortest path problem*** *asks for a directed path from s to t of minimum length.*
- *Given a node s, the* ***source****, the* ***single-source shortest path problem*** *asks for a shortest directed path from s to every node $j \in N$.*

The minimum length of a directed path from s to j is the ***distance*** *$\tilde{d}(j)$ of j; we set $\tilde{d}(j) = +\infty$ if j is not reachable from s.*

Notation

- In this chapter we write

$$n = |N|, \qquad m = |A|,$$

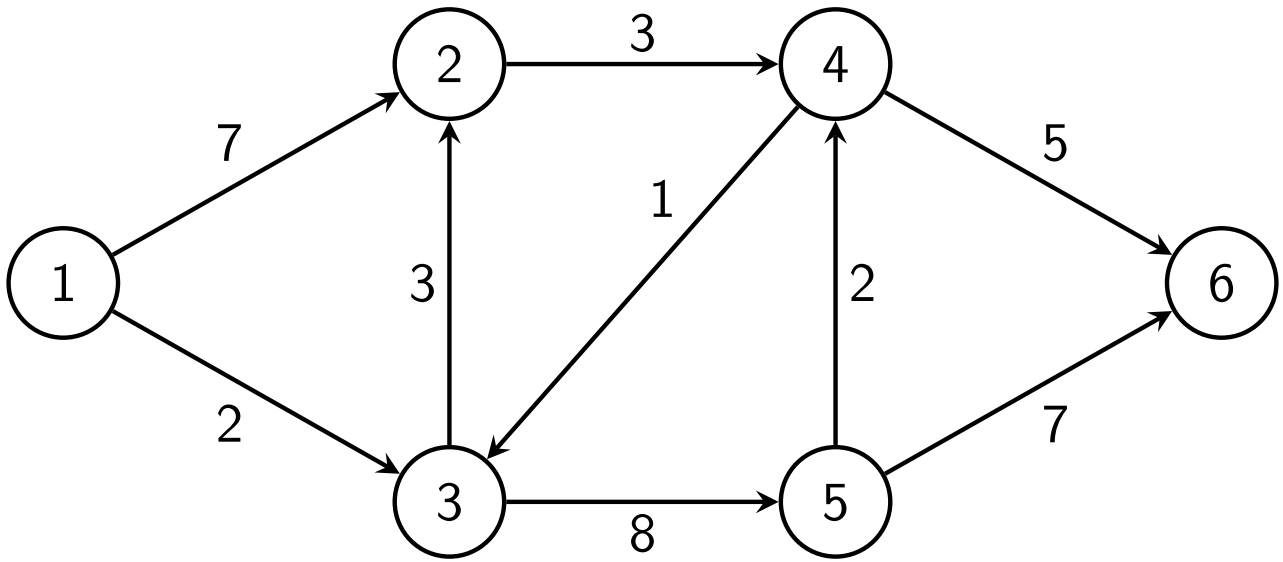
  and "path" and "cycle" always mean directed path and directed cycle.

- An algorithm for the single-source problem also solves the s–t problem, for every t at once.

- Let us now give lengths to the reference graph of the course: we will use it in every example of the chapter.

Example (1/2)

We give the reference graph of the chapter «Graphs» the lengths d_{ij} written on the arcs, all non-negative; the source is $s = 1$.



Example (2/2)

- Three of the paths from 1 to 6 and their lengths are:

$$d(1,2,4,6) = 7 + 3 + 5 = 15,$$

$$d(1,3,2,4,6) = 2 + 3 + 3 + 5 = 13,$$

$$d(1,3,5,6) = 2 + 8 + 7 = 17.$$

- The distances from node 1, computed later with Dijkstra's algorithm, are

$$\tilde{d} = (0,\ 5,\ 2,\ 8,\ 10,\ 13),$$

so $1, 3, 2, 4, 6$ is a shortest path from 1 to 6.

The lengths may be negative.

What can go wrong? A cycle whose arcs add up to a negative length.

Definition 2: negative cycle

*A **negative cycle** is a directed cycle C with $d(C) < 0$. In the rest of the chapter we refer to two assumptions:*

(A1) *every node of G is reachable from s;*

(A2) *no negative cycle of G is reachable from s.*

Lemma 1: walks and paths

Under (A2), for every directed walk W from s to j there is a directed path P from s to j, with arcs of W, such that $d(P) \leq d(W)$.

Proof

By induction on the number of arcs of W. If W has distinct nodes, $P = W$.

1. Otherwise W repeats a node: $W = s, \ldots, i_a, \ldots, i_b, \ldots, j$ with $i_a = i_b$, $a < b$, and $b - a$ minimum. The piece $C = i_a, \ldots, i_b$ is a cycle, reachable from s along W.

2. By (A2), $d(C) \geq 0$. Deleting C from W gives a walk W' from s to j with fewer arcs and

$$d(W') = d(W) - d(C) \leq d(W).$$

3. By induction W' contains a path P with

$$d(P) \leq d(W') \leq d(W).$$

□

Walks and paths

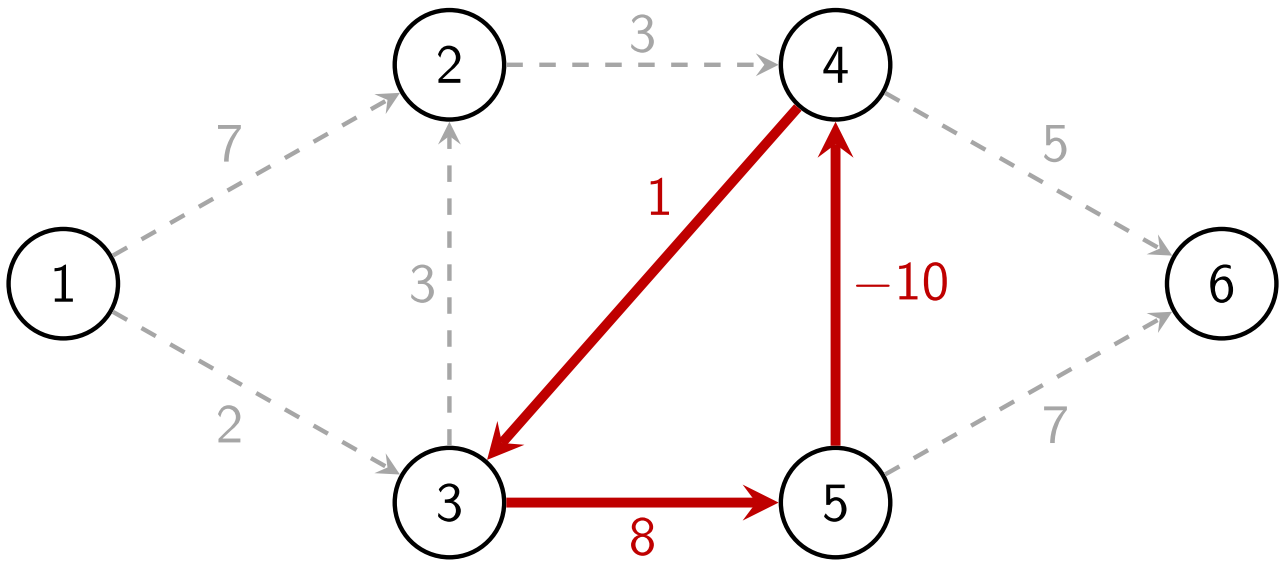
- Under (A2) the minimum length of a walk from s to j equals the distance $\tilde{d}(j)$: the algorithms of this chapter compute lengths of walks, and the lemma turns them into paths.
- With a negative cycle C reachable from s and from which j is reachable, going around C again and again gives walks from s to j of arbitrarily negative length.
- The algorithms of this chapter then detect C instead of computing the distances.

Example (1/2)

- The reference graph has two cycles, $2, 4, 3, 2$ and $3, 5, 4, 3$. With the lengths of the first example their lengths are 7 and 11.

- If the length of the arc $(5, 4)$ drops from 2 to $d''_{54} = -10$, the cycle $3, 5, 4, 3$ (red) has length

$$8 - 10 + 1 = -1.$$



Example (2/2)

- The walks from 1 to 6

$$1, 3, 5, 6,$$

$$1, 3, 5, 4, 3, 5, 6,$$

$$1, 3, 5, 4, 3, 5, 4, 3, 5, 6, \ \ldots$$

  have lengths $17, 16, 15, \ldots$: no walk is shortest.

- We use these lengths d'' again with the Bellman–Ford algorithm.

Without negative cycles, shortest paths have a simple structure:

every piece of a shortest path is itself shortest.

Lemma 2: optimality of subpaths

Under (A2), let P be a shortest path from s to j, and let i be a node of P. Then the subpath P_{si} of P from s to i is a shortest path from s to i.

Proof

Write P as P_{si} followed by the subpath P_{ij} from i to j, so

$$d(P) = d(P_{si}) + d(P_{ij}).$$

Assume by contradiction that a path Q from s to i has $d(Q) < d(P_{si})$.

1. Q followed by P_{ij} is a walk from s to j of length

$$d(Q) + d(P_{ij}) < d(P).$$

2. By the lemma on walks and paths it contains a path from s to j shorter than P: a contradiction.

□

Applied to the last arc of a shortest path, the optimality of subpaths
gives a system of equations satisfied by the distances.

Theorem 1: Bellman's equations

Under (A1) and (A2), the distances satisfy

$$\tilde{d}(s) = 0, \qquad \tilde{d}(j) = \min_{(i,j)\in\delta^-(j)} \left\{ \tilde{d}(i) + d_{ij} \right\}, \quad \forall j \in N \setminus \{s\},$$

and $\tilde{d}(j) \leq \tilde{d}(i) + d_{ij}$ *for every arc* $(i,j) \in A$.

Proof (1/2)

- All distances are finite by (A1).
- $\tilde{d}(s) = 0$: the path s has length 0, and a path from s to s with arcs would be a cycle, of length ≥ 0 by (A2).

Proof (2/2)

- *Inequality.* A shortest path from s to i followed by the arc (i, j) is a walk of length $\tilde{d}(i) + d_{ij}$; by the lemma on walks and paths,
$$\tilde{d}(j) \leq \tilde{d}(i) + d_{ij}.$$
- *Equation.* For $j \neq s$, let P be a shortest path from s to j and (i, j) its last arc. By the optimality of subpaths,
$$\tilde{d}(j) = d(P) = \tilde{d}(i) + d_{ij},$$
so the minimum is attained.

□

Proposition 1: shortest path tree

*Under (A1) and (A2), there is a spanning tree T of G, the **shortest path tree**, such that for every $j \in N$ the path from s to j in T is directed and is a shortest path.*

- *Approach of the proof:* for every node we keep the last arc of a shortest path with the fewest arcs, and we show that these arcs form a tree.

Proof (1/2)

For every j, let $k(j)$ be the minimum number of arcs of a shortest path from s to j; $k(s) = 0$.

1. For $j \neq s$, choose a shortest path P from s to j with $k(j)$ arcs, and let $pred(j)$ be the node before j on P.

2. The subpath of P from s to $pred(j)$ is shortest (optimality of subpaths) with $k(j) - 1$ arcs, so

$$k(pred(j)) \leq k(j) - 1 \qquad \text{and} \qquad \tilde{d}(j) = \tilde{d}(pred(j)) + d_{pred(j)\,j}.$$

Proof (2/2)

3. Moving from j to $pred(j)$, the number k strictly decreases, so the sequence ends in s. The arcs $(pred(j), j)$ are $n-1$ and connect every node to s: they form a spanning tree (chapter «Graphs», Theorem «characterizations of a tree»).

4. Along the tree path $s = i_0, i_1, \ldots, i_r = j$ every arc satisfies

$$\tilde{d}(i_t) = \tilde{d}(i_{t-1}) + d_{i_{t-1} i_t};$$

adding up, its length is $\tilde{d}(j)$.

□

Example (1/2)

Bellman's equations on the reference graph, with the distances $\tilde{d} = (0, 5, 2, 8, 10, 13)$ of the first example:

- node 4 has the entering arcs $(2, 4)$ and $(5, 4)$, and

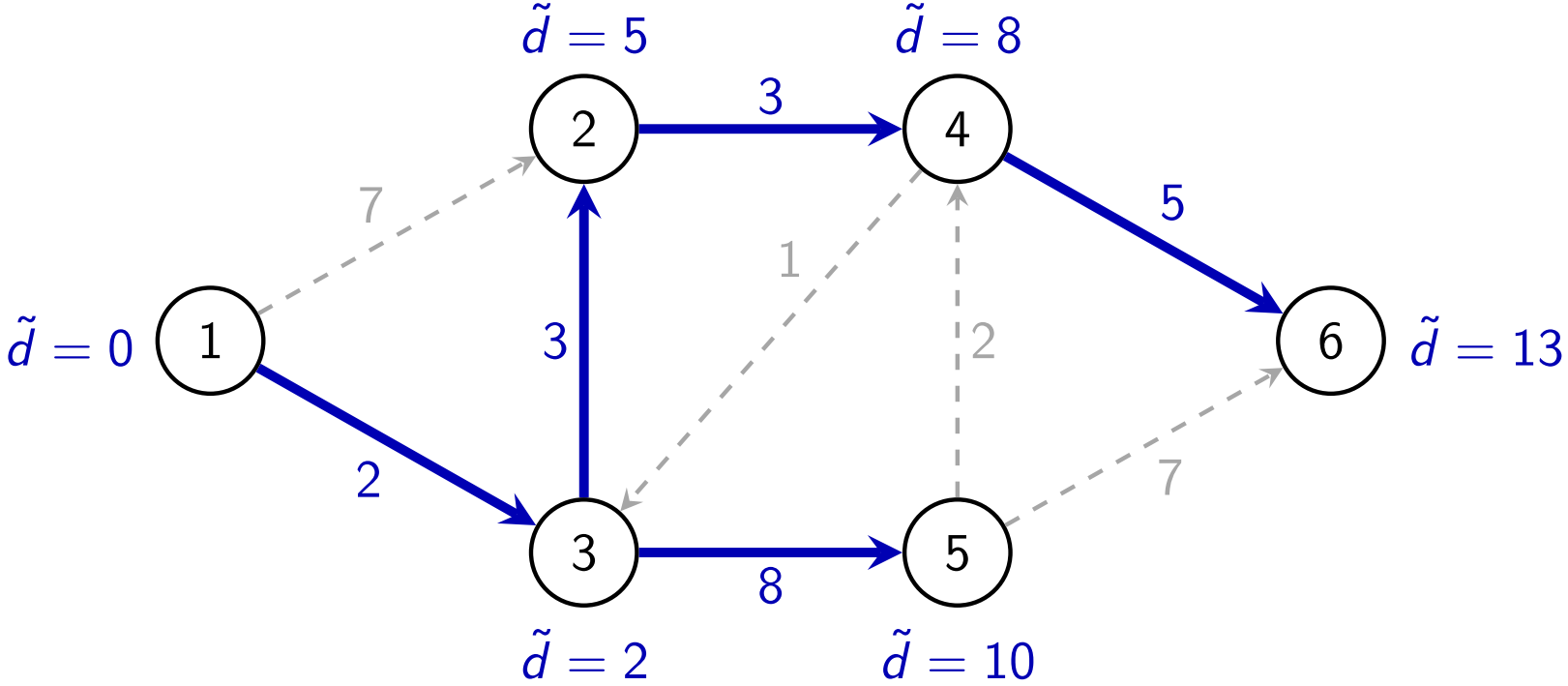
$$\tilde{d}(4) = \min\{5 + 3,\ 10 + 2\} = 8;$$

- node 6 has the entering arcs $(4, 6)$ and $(5, 6)$, and

$$\tilde{d}(6) = \min\{8 + 5,\ 10 + 7\} = 13.$$

Example (2/2)

The arcs attaining the minimum form the shortest path tree $(1,3)$, $(3,2)$, $(2,4)$, $(3,5)$, $(4,6)$, in blue; next to each node its distance.



Summary

Shortest path problems:

- s–t and single-source problems; the distance $\tilde{d}(j)$ is the length of a shortest path
- A negative cycle reachable from s makes the walks arbitrarily short: assumptions (A1) and (A2)

Structure of shortest paths:

- Without negative cycles reachable from the source, every subpath of a shortest path is shortest
- The distances satisfy Bellman's equations
- One spanning tree contains a shortest path to every node

Optimality conditions and potentials

Introduction: Optimality conditions and potentials

- Someone gives us a path and claims it is shortest: how can we check it without enumerating all paths?
- We give a number, the **potential**, to every node, and look at the **reduced costs** of the arcs
- Feasible potentials are a **certificate of optimality**; they will turn out to be the dual variables of the LP model of the next section

Definition 3: potentials and reduced costs

A vector $\boldsymbol{\pi} \in \mathbb{R}^n$ *gives a* ***potential*** π_i *to every node* $i \in N$. *The* ***reduced cost*** *of the arc* (i, j) *with respect to* $\boldsymbol{\pi}$ *is*

$$\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j.$$

The potentials are ***feasible*** *if* $\bar{c}_{ij} \geq 0$ *for every arc* $(i, j) \in A$, *and an arc with* $\bar{c}_{ij} = 0$ *is* ***tight***.

Labels and potentials

- Given **labels** $d(j)$, lengths of walks from s to j, we use the potentials $\pi_i = -d(i)$. Then

$$\bar{c}_{ij} = d_{ij} + d(i) - d(j),$$

and the potentials are feasible if and only if $d(j) \leq d(i) + d_{ij}$ for every arc.

- Along a walk $W = i_0, i_1, \ldots, i_k$ the potentials telescope:

$$d(W) = \sum_{t=1}^{k} \bar{c}_{i_{t-1} i_t} \; + \; \pi_{i_0} - \pi_{i_k}.$$

Theorem 2: potentials as optimality certificate

Let π be feasible potentials, and let P be a path from s to t whose arcs are all tight. Then P is a shortest path from s to t, and $d(P) = \pi_s - \pi_t$. Moreover, G has no negative cycle.

Proof

We use the telescoping formula above.

1. The arcs of P have $\bar{c}_{ij} = 0$, so $d(P) = \pi_s - \pi_t$.
2. For every path Q from s to t,

$$d(Q) = \sum_{(i,j)\in Q} \bar{c}_{ij} + \pi_s - \pi_t \geq \pi_s - \pi_t = d(P).$$

3. For every cycle C the first and the last node coincide, so

$$d(C) = \sum_{(i,j)\in C} \bar{c}_{ij} \geq 0.$$

□

Corollary 1: optimality conditions for labels

Let the labels $d(j)$, $j \in N$, be finite with $d(s) = 0$, and assume that

(i) $d(j) \leq d(i) + d_{ij}$ *for every arc* $(i, j) \in A$;

(ii) *for every* $j \neq s$ *there is a path from* s *to* j *of length* $d(j)$.

Then $d(j) = \tilde{d}(j)$ *for every* $j \in N$.

Proof

- By (i) the potentials $\pi_i = -d(i)$ are feasible, so every path Q from s to j has

$$d(Q) \geq \pi_s - \pi_j = d(j)$$

by the argument of the theorem on potentials as optimality certificate.

- By (ii) the value $d(j)$ is attained by a path, so $d(j) = \tilde{d}(j)$.

□

Example (1/2)

A certificate on the reference graph: with the distances of the first example the potentials are

$$\boldsymbol{\pi} = -\tilde{\boldsymbol{d}} = (0, -5, -2, -8, -10, -13),$$

and the reduced costs $\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j$ are:

arc	$(1,2)$	$(1,3)$	$(2,4)$	$(3,2)$	$(3,5)$	$(4,3)$	$(4,6)$	$(5,4)$	$(5,6)$
d_{ij}	7	2	3	3	8	1	5	2	7
$\bar{c}_{ij}$	2	0	0	0	0	7	0	4	4

Example (2/2)

- All reduced costs are non-negative, and the tight arcs are the arcs of the shortest path tree. The path $1, 3, 2, 4, 6$ is tight, so it is shortest, with length $\pi_1 - \pi_6 = 13$.

- The tree of the breadth-first search, $(1, 2)$, $(1, 3)$, $(2, 4)$, $(3, 5)$, $(4, 6)$, gives the labels $(0, 7, 2, 10, 10, 15)$. With their potentials,

$$\bar{c}_{32} = 3 - (-2) + (-7) = -2 < 0 :$$

the potentials are not feasible, and entering node 2 through $(3, 2)$ shortens its path by 2.

Summary

Potentials:

- Reduced costs $\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j$; feasible potentials have $\bar{c}_{ij} \geq 0$ on every arc
- Feasible potentials and a path made of tight arcs prove that the path is shortest

Labels:

- With labels, $\pi_i = -d(i)$, and feasibility reads $d(j) \leq d(i) + d_{ij}$
- Labels that satisfy it and are lengths of paths are the distances

The linear programming formulation

Introduction: The linear programming formulation

- A path from s to t can be seen as **one unit of flow** sent from s to t
- This gives an LP model with the node–arc incidence matrix; its dual is exactly the problem of the feasible potentials
- The same flow model, with other balances and capacities, is the starting point of the chapters on flows later in the course

A path as a unit flow

- A path from s to t sends one unit of flow from s to t.
- We use a variable $x_{ij} \geq 0$ for every arc, the flow on the arc, and the **balances**

$$b_s = 1, \qquad b_t = -1, \qquad b_i = 0, \quad \forall i \in N \setminus \{s, t\}.$$

The $\mathrm{SP}(s,t)$ model

Using these variables, an LP model for the s–t shortest path problem reads as follows:

$$\min \sum_{(i,j)\in A} d_{ij}\, x_{ij}$$

$$\text{subject to} \sum_{(i,j)\in\delta^+(i)} x_{ij} - \sum_{(j,i)\in\delta^-(i)} x_{ji} = b_i, \quad \forall i \in N,$$

$$x_{ij} \geq 0, \quad \forall (i,j) \in A.$$

This LP model is referred to as the $\mathrm{SP}(s,t)$ model.

Description of the $\mathrm{SP}(s, t)$ model

Description of the objective function and constraints of the $\mathrm{SP}(s, t)$ model:

- the objective function minimizes the total length of the flow;
- the equality constraints are the flow conservation constraints: one unit leaves s, one unit enters t, and every other node sends out what it receives;
- finally, the non-negativity constraints define the variables of the LP model.

Matrix form and incidence vectors

- In matrix form the model is

$$\min\{\boldsymbol{d}'\boldsymbol{x} : \boldsymbol{B}\boldsymbol{x} = \boldsymbol{b},\ \boldsymbol{x} \geq 0\},$$

  with the node–arc incidence matrix $\boldsymbol{B}$ of the chapter «Graphs»: $+1$ in the tail and -1 in the head of every arc.

- The **incidence vector** $\boldsymbol{x}^P$ of a path P from s to t, with $x^P_{ij} = 1$ on the arcs of P and 0 elsewhere, is feasible, and its value is $d(P)$.

The dual variables

- The constraint of node i is an equation, so its dual variable π_i is free.
- The column of x_{ij} in $\boldsymbol{B}$ has $+1$ in row i and -1 in row j, so the dual constraint of $x_{ij} \geq 0$ is

$$\pi_i - \pi_j \leq d_{ij}$$

(module Linear Programming, chapter «Duality theory»).

The $D(\mathrm{SP}(s,t))$ model

Using a free dual variable π_i for each flow conservation constraint, the dual model of the $\mathrm{SP}(s,t)$ model is:

$$\begin{aligned} \max \quad & \pi_s - \pi_t \\ \text{subject to} \quad & \pi_i - \pi_j \leq d_{ij}, \quad \forall (i,j) \in A, \\ & \pi_i \in \mathbb{R}, \quad \forall i \in N. \end{aligned}$$

This LP model is referred to as the $D(\mathrm{SP}(s,t))$ model.

Dual solutions and potentials

- The dual constraint is

$$\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j \geq 0 :$$

  the feasible solutions of the dual are the feasible potentials of the previous section.

- $\bar{c}_{ij}$ is the reduced cost of x_{ij} in the LP sense.

Theorem 3: LP and shortest paths

Under (A1) and (A2), the $\mathrm{SP}(s,t)$ *model has an optimal solution and* $z(\mathrm{SP}(s,t)) = \tilde{d}(t)$. *The incidence vector of a shortest path from* s *to* t *is optimal for* $\mathrm{SP}(s,t)$, *and the potentials* $\pi_i = -\tilde{d}(i)$ *are optimal for* $D(\mathrm{SP}(s,t))$.

Proof

Let P be a shortest path from s to t, which exists by (A1).

1. $\boldsymbol{x}^P$ is feasible for $\mathrm{SP}(s,t)$, with value $d(P) = \tilde{d}(t)$.
2. By Bellman's equations, $\tilde{d}(j) \leq \tilde{d}(i) + d_{ij}$ for every arc, so $\pi_i = -\tilde{d}(i)$ satisfies $\pi_i - \pi_j \leq d_{ij}$; its value is
$$\pi_s - \pi_t = 0 + \tilde{d}(t).$$
3. A feasible primal and a feasible dual solution with the same value are both optimal, by weak duality.

□

Proposition 2: a negative cycle makes the model unbounded

If $\mathrm{SP}(s,t)$ *is feasible and* G *has a negative cycle* C*, reachable from* s *or not, then* $\mathrm{SP}(s,t)$ *is unbounded.*

Proof

- Let $\bar{\boldsymbol{x}}$ be feasible and let $\boldsymbol{x}^C$ be the incidence vector of C.
- The columns of the arcs of C add up to zero (chapter «Graphs», Lemma «columns of a cycle»), so $\boldsymbol{B}\boldsymbol{x}^C = 0$.
- For every $\lambda \geq 0$ the vector $\bar{\boldsymbol{x}} + \lambda \boldsymbol{x}^C$ is feasible, with value

$$\boldsymbol{d}'\bar{\boldsymbol{x}} + \lambda\, d(C),$$

which tends to $-\infty$.

□

Observation 1: The sign of the potentials

With the incidence matrix of the course ($+1$ in the tail) the dual constraint is $\pi_i - \pi_j \leq d_{ij}$, and the optimal potentials are $\pi_i = -\tilde{d}(i)$, up to an additive constant. The form $\pi_j - \pi_i \leq d_{ij}$, with $\pi_i = \tilde{d}(i)$, belongs to the opposite sign convention (-1 in the tail).

The single-source model

- To find a shortest path from s to every node, s sends one unit to each of the other $n-1$ nodes:

$$b_s = n-1, \qquad b_i = -1, \quad \forall i \in N \setminus \{s\}.$$

- With these balances the $\mathrm{SP}(s,t)$ model is called $\mathrm{SP}(s)$.

- Its dual maximizes $(n-1)\pi_s - \sum_{i \neq s} \pi_i$ under the same constraints $\pi_i - \pi_j \leq d_{ij}$.

Proposition 3: the single-source model

Under (A1) and (A2), let T be a shortest path tree and, for every $j \neq s$, let T_j be the set of the nodes whose tree path passes through j (j included). The flow with $x_{ij} = |T_j|$ on the arcs (i, j) of T and 0 elsewhere is optimal for $\mathrm{SP}(s)$, the potentials $\pi_i = -\tilde{d}(i)$ are optimal for its dual, and

$$z(\mathrm{SP}(s)) = \sum_{j \in N} \tilde{d}(j).$$

Proof

1. *Feasibility.* At a node $j \neq s$ the flow entering on the tree arc is $|T_j|$, and the flow leaving is $\sum |T_h|$ over the tree arcs (j, h), that is $|T_j| - 1$: the balance is -1. At s the leaving flow covers all the other nodes: $n - 1$.

2. *Value.* The flow is the sum of the incidence vectors of the $n - 1$ tree paths, since the arc (i, j) lies on the tree paths of the $|T_j|$ nodes of T_j. Its value is $\sum_j \tilde{d}(j)$.

3. *Dual.* $\pi_i = -\tilde{d}(i)$ is feasible as in the theorem on LP and shortest paths, with value $(n-1) \cdot 0 + \sum_{i \neq s} \tilde{d}(i)$. By weak duality both solutions are optimal.

□

Integral vertices

- In the chapter «Network flow models» the vertices of these polyhedra are shown to be integral, by the total unimodularity of $\boldsymbol{B}$.
- Let us now solve the two models on the reference graph.

Example (1/3)

We solve the models with the lengths of the first example, $s = 1$ and $t = 6$ (values checked with Gurobi).

- $\mathrm{SP}(1, 6)$: $z(\mathrm{SP}(1, 6)) = 13$, with

$$x_{13} = x_{32} = x_{24} = x_{46} = 1$$

  and all other variables 0: the path $1, 3, 2, 4, 6$.

- The dual value is $\pi_1 - \pi_6 = 13$.

Example (2/3)

- $\mathrm{SP}(1)$: $z(\mathrm{SP}(1)) = 0 + 5 + 2 + 8 + 10 + 13 = 38$, with the flows on the shortest path tree

$$x_{13} = 5, \qquad x_{32} = 3, \qquad x_{24} = 2, \qquad x_{46} = 1, \qquad x_{35} = 1.$$

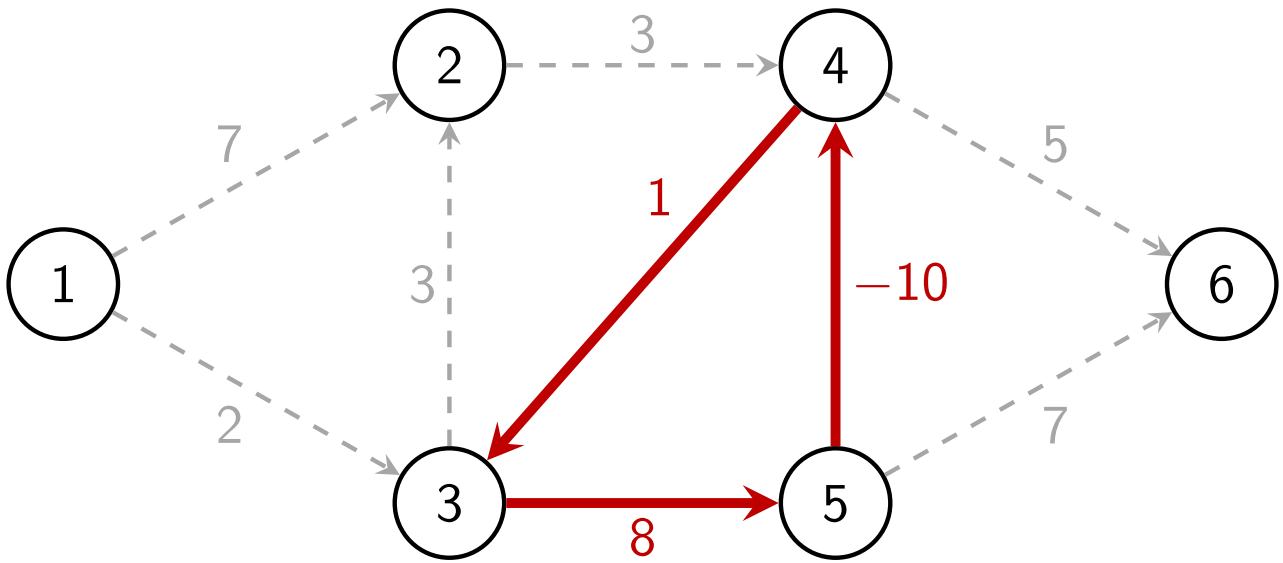
- For example $T_3 = \{2, 3, 4, 5, 6\}$, so $x_{13} = 5$; the cost is

$$2 \cdot 5 + 3 \cdot 3 + 3 \cdot 2 + 5 + 8 = 38.$$

- The optimal duals, normalized with $\pi_1 = 0$, are $\boldsymbol{\pi} = (0, -5, -2, -8, -10, -13)$.

Example (3/3)

- With the lengths d'' of the negative cycle example ($d''_{54} = -10$) the model $\mathrm{SP}(1, 6)$ is unbounded.
- Each unit of flow around the cycle $3, 5, 4, 3$ lowers the value by 1.



Summary

The primal model:

- A shortest path is a unit flow from s to t: $\min \boldsymbol{d}'\boldsymbol{x}$ with $\boldsymbol{B}\boldsymbol{x} = \boldsymbol{b}$, $\boldsymbol{x} \geq 0$
- A negative cycle anywhere in G makes the model unbounded

The dual model:

- Its constraints are $\pi_i - \pi_j \leq d_{ij}$: the feasible potentials
- $\pi_i = -\tilde{d}(i)$ is optimal, also for the single-source model $\mathrm{SP}(s)$

Shortest paths in acyclic graphs

Introduction: Shortest paths in acyclic graphs

- We start the algorithms with the easiest case: a graph without directed cycles
- Ordering the nodes so that every arc goes forward, one scan of the arcs gives all the distances, with lengths of any sign
- The same algorithm computes longest paths, the core of project management later in the course

Definition 4: acyclic graph and topological order

A directed graph without directed cycles is a ***directed acyclic graph*** *(DAG). A* ***topological order*** *of* $G = (N, A)$ *is an ordering* $v_1, v_2, \ldots, v_n$ *of its nodes such that every arc goes forward:*

$$(v_a, v_b) \in A \quad \Longrightarrow \quad a < b.$$

Proposition 4: topological orders

A directed graph has a topological order if and only if it is acyclic.

Proof

If G has a topological order, a cycle $i_0, i_1, \ldots, i_k = i_0$ would go forward on every arc and come back to its first node: impossible. Conversely, let G be acyclic; we argue by induction on n.

1. G has a node with no entering arc: otherwise, walking backwards along entering arcs from any node, after n steps a node repeats, and the walk contains a cycle.

2. Put such a node v_1 first. The graph without v_1 is acyclic, and by induction it has a topological order $v_2, \ldots, v_n$; no arc enters v_1, so $v_1, v_2, \ldots, v_n$ is a topological order of G.

□

Algorithm: Shortest paths in an acyclic graph

Input : an acyclic graph $G = (N, A)$ with lengths d_{ij} of any sign, a source $s \in N$

Output: the distances $d(j) = \tilde{d}(j)$ and the predecessors $pred(j)$

1 **Step 0 (order).** Find a topological order: repeatedly remove a node without entering arcs (the smallest index, if more than one).

2 Set $d(s) := 0$ and $d(j) := +\infty$ for $j \neq s$.

3 **Step 1 (labels).** For each node i in topological order with $d(i) < +\infty$, and each arc $(i, j) \in \delta^+(i)$:

4 if $d(i) + d_{ij} < d(j)$, set $d(j) := d(i) + d_{ij}$ and $pred(j) := i$.

5 STOP.

Theorem 4: correctness and running time on acyclic graphs

On an acyclic graph with lengths of any sign, the algorithm «Shortest paths in an acyclic graph» ends with $d(j) = \tilde{d}(j)$ for every $j \in N$, and it runs in $O(n + m)$ time.

- *Approach of the proof:* the nodes of a shortest path appear in topological order, so each of them is processed after its predecessor on the path.

Proof (1/2)

An acyclic graph has no cycle at all, so (A2) holds. Every finite label is the length of a walk from s, so $d(j) \geq \tilde{d}(j)$ by the lemma on walks and paths; labels only decrease.

1. Let j be reachable and $s = i_0, i_1, \ldots, i_k = j$ a shortest path; its nodes appear in this order in the topological order, since every arc goes forward.

2. By induction on t, after i_t is processed $d(i_t) \leq \tilde{d}(i_t)$: the path to i_{t-1} is shortest (optimality of subpaths), and processing i_{t-1} gives
$$d(i_t) \leq \tilde{d}(i_{t-1}) + d_{i_{t-1} i_t} = \tilde{d}(i_t).$$

Proof (2/2)

3. Nodes not reachable from s keep $d(j) = +\infty = \tilde{d}(j)$.

Time.

- The topological order takes $O(n + m)$ time with a counter of the entering arcs of every node, decreased when a tail is removed.
- Step 1 scans every arc once.

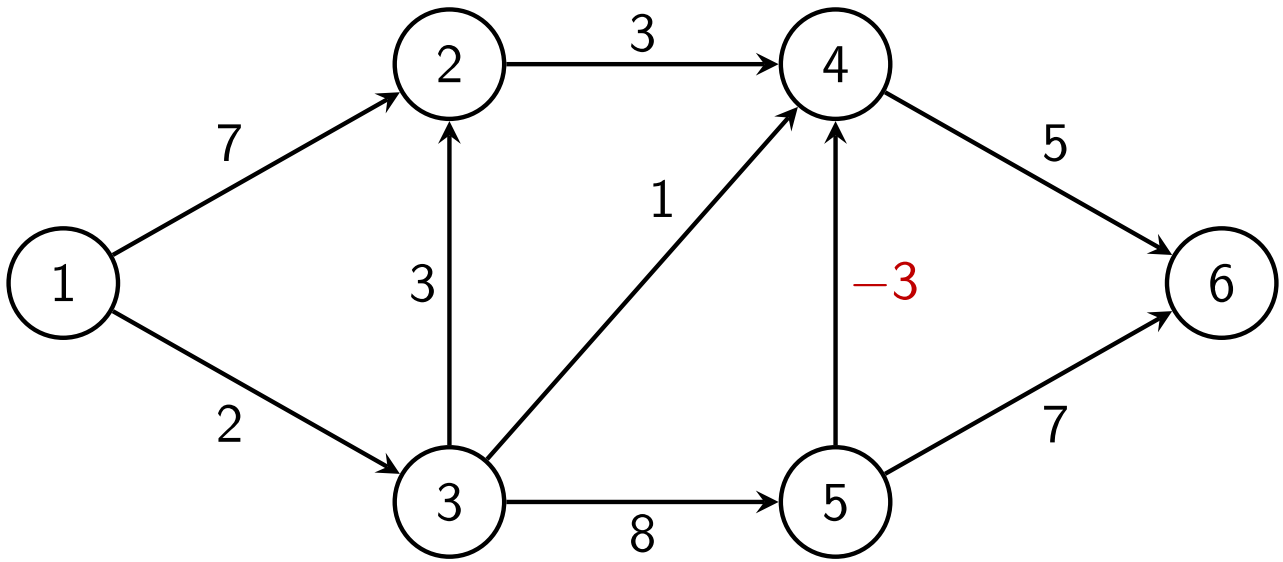
□

Remarks

- The smallest-index rule fixes one order for the examples; any node without entering arcs can be removed.
- With the lengths $-d_{ij}$ the algorithm computes **longest paths**, since an acyclic graph has no cycle to repeat.
- Longest paths in acyclic graphs are the core of the chapter «Project management».

Example: an acyclic instance

Reversing the arc $(4, 3)$ of the reference graph into $(3, 4)$, with $d_{34} = 1$, gives an acyclic graph. We give it the lengths of the first example except $d_{54} = -3$ (red).



Example: Step 0

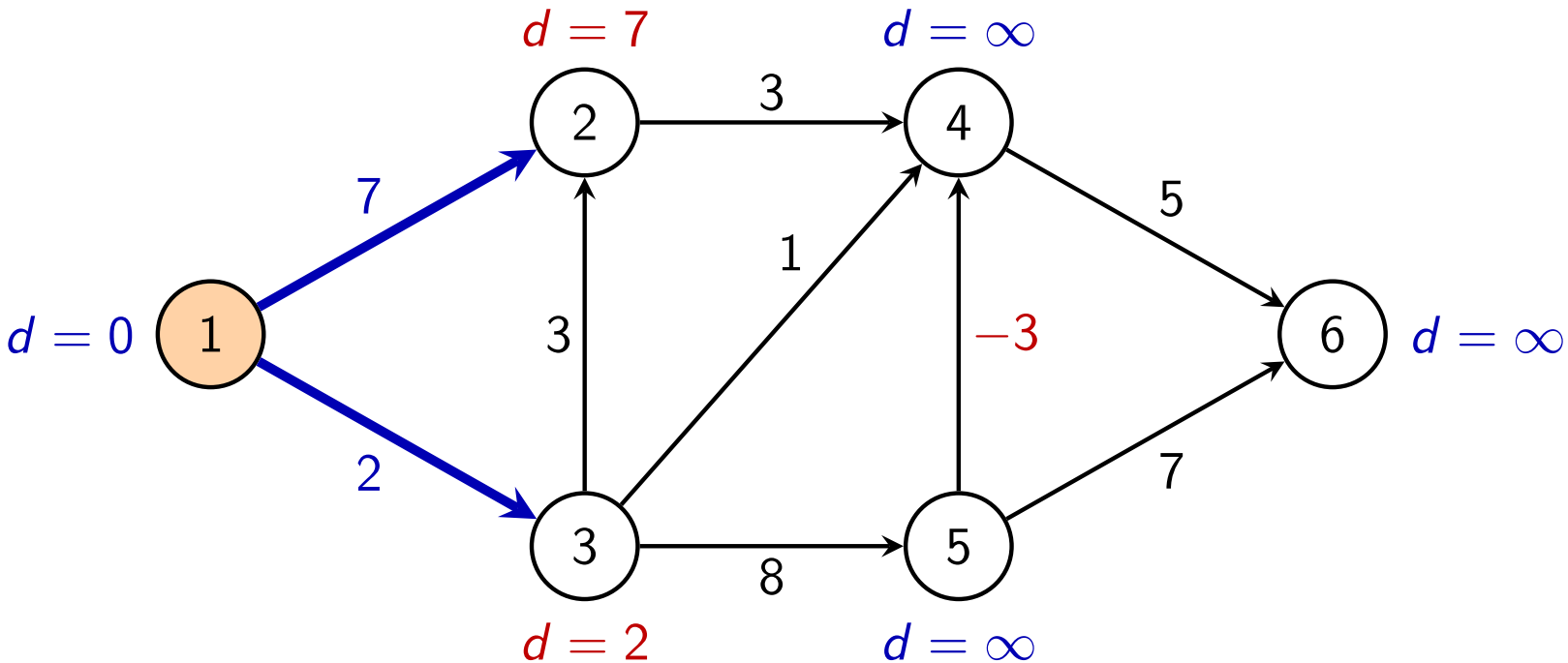
- Step 0 gives the topological order

$$1,\ 3,\ 2,\ 5,\ 4,\ 6.$$

- Only node 1 has no entering arc; after 1, only 3 has no entering arc.
- Then 2 and 5 are free, and 2 goes first (smallest index); then 5, 4 and 6.
- $d(1) = 0$ and $d(j) = +\infty$ for $j \neq 1$.
- In the next slides the node being processed is in orange, the processed nodes are shaded, a label that has just changed is in red, and the arcs $(pred(j), j)$ are in blue.

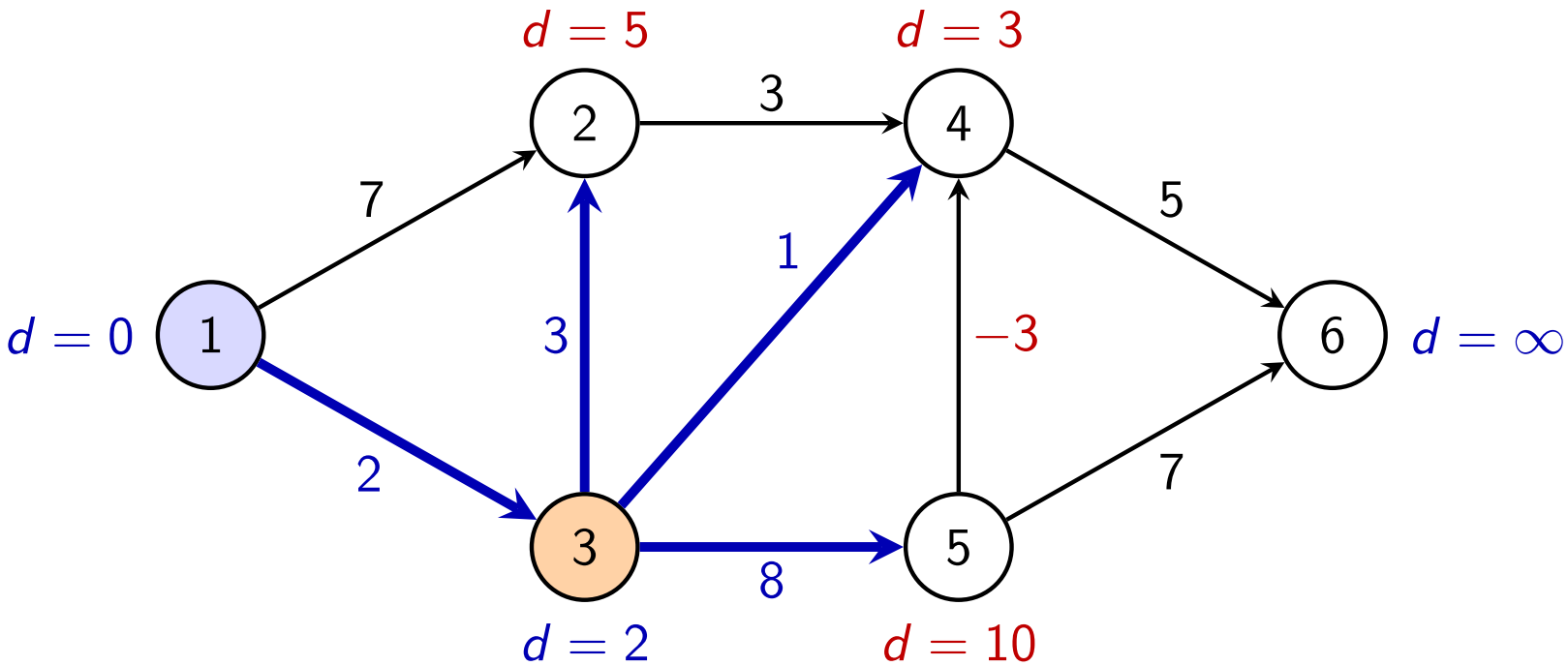
Example: step 1, node 1

- $(1,2)$: $0+7=7<\infty$, so $d(2):=7$, $pred(2):=1$;
- $(1,3)$: $0+2=2<\infty$, so $d(3):=2$, $pred(3):=1$.



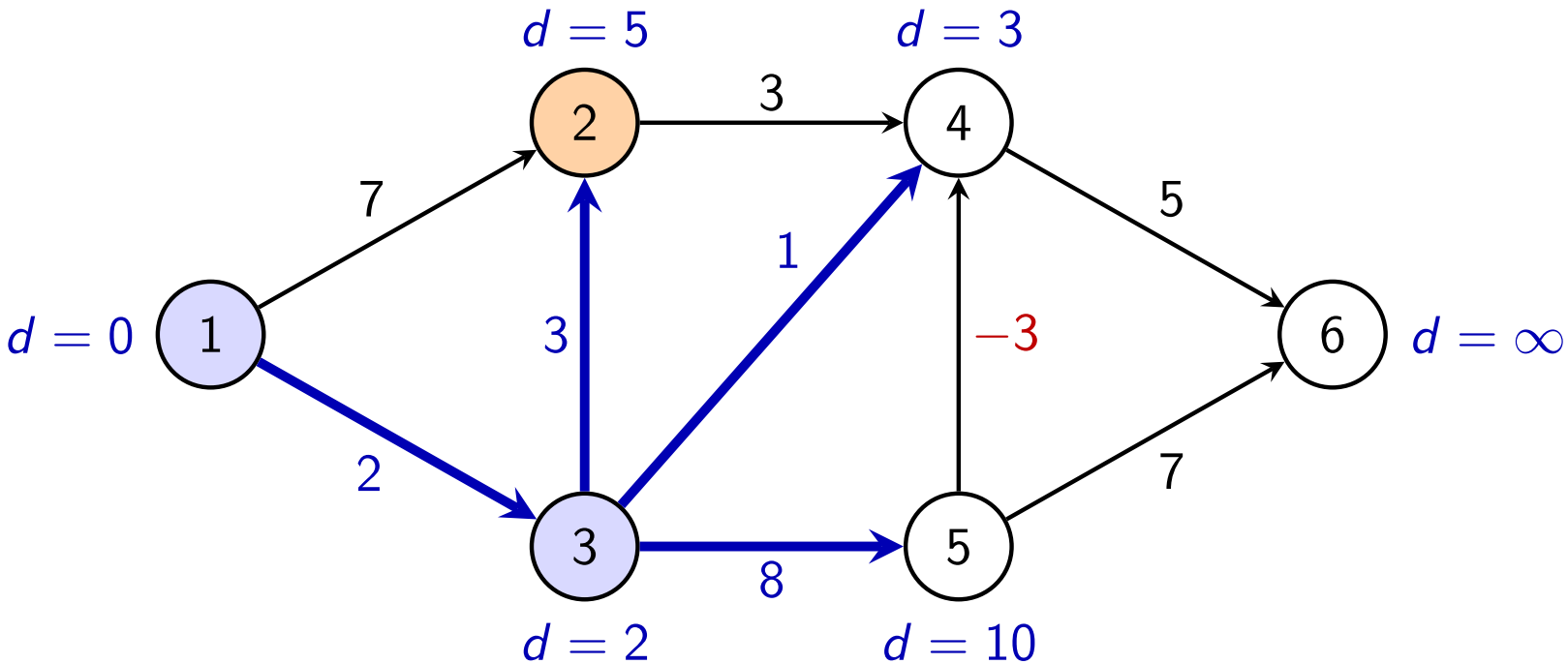
Example: step 2, node 3

- $(3, 2)$: $2 + 3 = 5 < 7$, so $d(2) := 5$, $pred(2) := 3$;
- $(3, 4)$: $2 + 1 = 3 < \infty$, so $d(4) := 3$, $pred(4) := 3$;
- $(3, 5)$: $2 + 8 = 10 < \infty$, so $d(5) := 10$, $pred(5) := 3$.



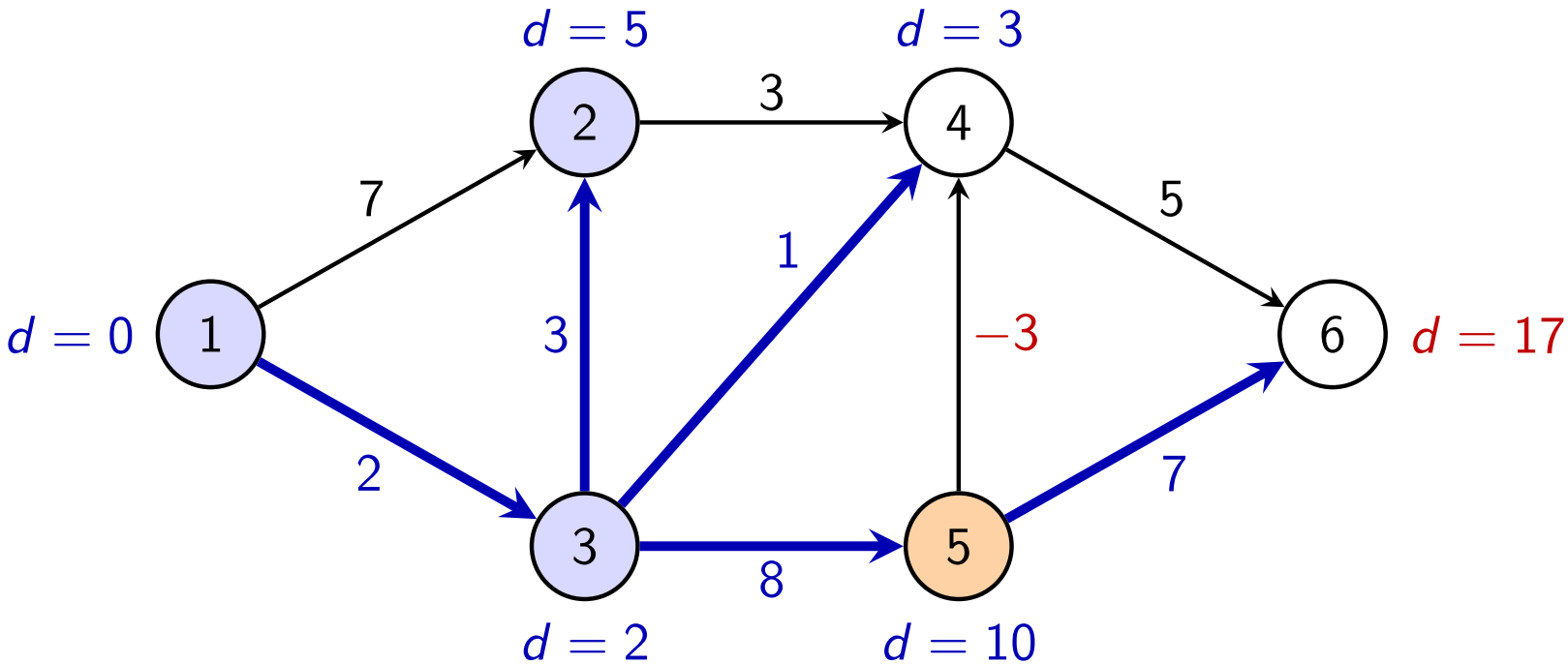
Example: step 3, node 2

- $(2,4)$: $5+3=8 \geq 3 = d(4)$: no change.



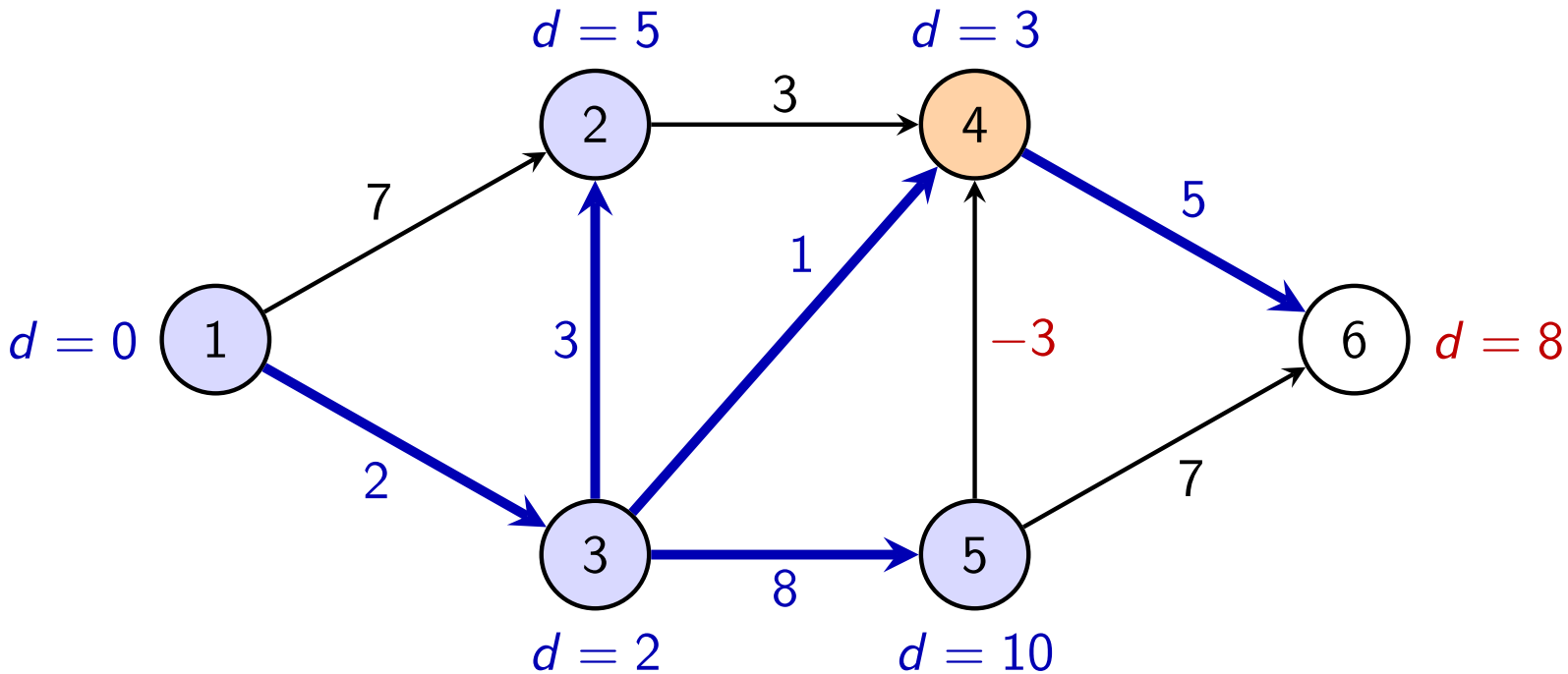
Example: step 4, node 5

- $(5,4)$: $10-3=7\geq 3=d(4)$: no change;
- $(5,6)$: $10+7=17<\infty$, so $d(6):=17$, $pred(6):=5$.



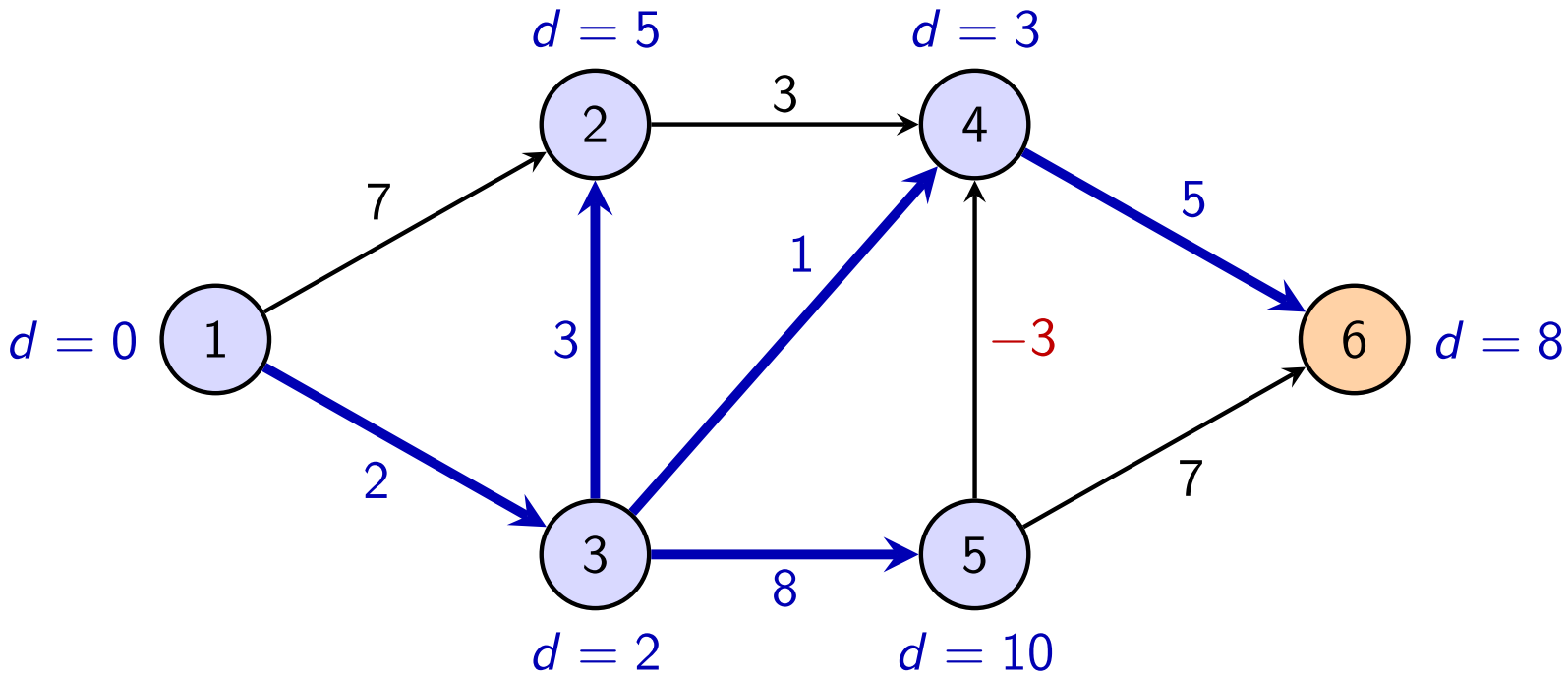
Example: step 5, node 4

- $(4, 6)$: $3 + 5 = 8 < 17$, so $d(6) := 8$, $pred(6) := 4$.



Example: step 6, node 6

- Node 6 has no leaving arc: nothing changes, and the algorithm stops.



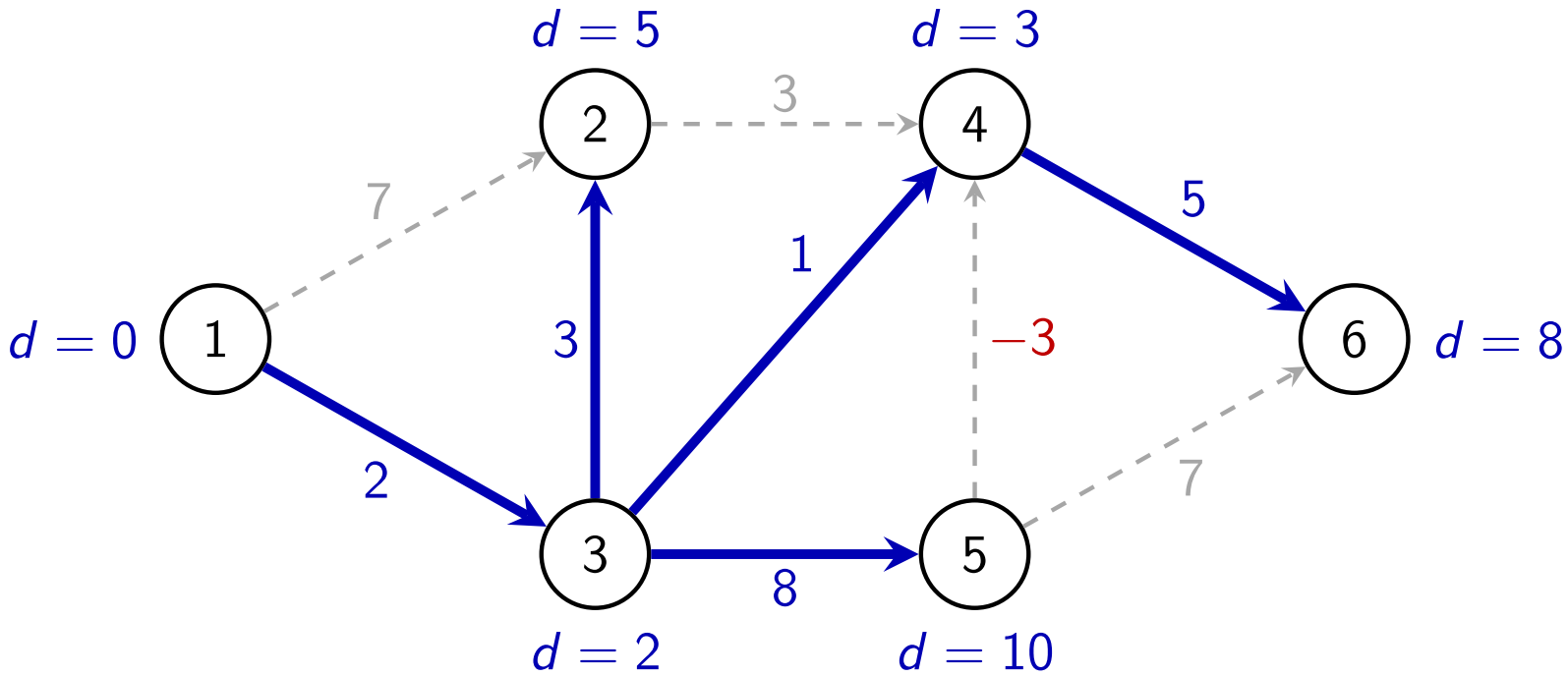
Example: the table of the labels

Each row of the table processes one node and shows the labels after it:

step	node i	$d(1)$	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$
1	1	0	7	2	∞	∞	∞
2	3	0	5	2	3	10	∞
3	2	0	5	2	3	10	∞
4	5	0	5	2	3	10	17
5	4	0	5	2	3	10	8
6	6	0	5	2	3	10	8

Example: the result

The predecessors give the tree $(1,3)$, $(3,2)$, $(3,4)$, $(3,5)$, $(4,6)$ (blue).



- The negative arc $(5,4)$ is not used: $d(5) - 3 = 7 > 3 = d(4)$.
- With the lengths $-d_{ij}$ the longest path lengths from 1 are $(0, 7, 2, 10, 10, 17)$.

Summary

Topological order:

- A graph is acyclic if and only if it has a topological order
- It is found by repeatedly removing a node without entering arcs

The algorithm:

- Scanning the nodes in topological order, each label is final when its node is processed
- Shortest paths in $O(n + m)$ time, with lengths of any sign; with $-d_{ij}$, longest paths

The Bellman–Ford algorithm

Introduction: The Bellman–Ford algorithm

- Now the graph may have cycles and the lengths may be negative
- The Bellman–Ford algorithm computes the shortest walks with at most $1, 2, \ldots$ arcs, one pass at a time
- It finds the distances, or it detects a negative cycle reachable from the source: the general tool when some length is negative

Definition 5: labels with at most k arcs

For $k \geq 0$ and $j \in N$, $d^k(j)$ is the minimum length of a walk from s to j with at most k arcs, and $d^k(j) = +\infty$ if there is none.

Lemma 3: recursion of the labels

$d^0(s) = 0$, $d^0(j) = +\infty$ *for* $j \neq s$, *and for every* $k \geq 1$

$$d^k(j) = \min \left\{ d^{k-1}(j), \min_{(i,j) \in \delta^-(j)} \left(d^{k-1}(i) + d_{ij} \right) \right\}, \quad \forall j \in N.$$

Proof

- The only walk with 0 arcs is the walk s.
- For $k \geq 1$, a walk to j with at most k arcs either
  - has at most $k-1$ arcs, with minimum length $d^{k-1}(j)$, or
  - has exactly k arcs and last arc (i,j): it is a walk to i with $k-1$ arcs followed by (i,j), and its minimum length is

$$\min_{(i,j)} \left(d^{k-1}(i) + d_{ij} \right).$$

□

Algorithm: Bellman–Ford algorithm

Input : a directed graph $G = (N, A)$ with lengths d_{ij} of any sign, a source $s \in N$

Output: the distances $\tilde{d}(j)$ and the predecessors, or a negative cycle reachable from s

1 **Step 0 (initialization).** Set $d^0(s) := 0$, $d^0(j) := +\infty$ for $j \neq s$, and $k := 1$.

2 **Step 1 (pass k).** Compute $d^k(j)$ for every $j \in N$ with the recursion of the labels;

3 if $d^k(j) < d^{k-1}(j)$, set $pred(j)$ to the tail i of the arc that attains the minimum.

4 **Step 2 (test).** If $d^k = d^{k-1}$, STOP: $\tilde{d} = d^k$.

5 If $k = n$, STOP: a negative cycle is reachable from s. Otherwise set $k := k + 1$ and go to Step 1.

How the passes work

- Each pass depends only on the labels of the previous pass: if $d^k = d^{k-1}$, all the following passes repeat the same labels, and the algorithm can stop.
- In the examples, ties in the minimum are broken by the smallest tail i, and the predecessor changes only when the label strictly decreases.

Theorem 5: correctness of Bellman–Ford

If no negative cycle is reachable from s, then $d^{n-1}(j) = \tilde{d}(j)$ for every $j \in N$, and the algorithm stops at a pass $k \leq n$ with $d^k = d^{k-1} = \tilde{d}$.

Proof

- A path has at most $n-1$ arcs, so $d^{n-1}(j) \leq \tilde{d}(j)$.
- By the lemma on walks and paths (which needs only the cycles reachable from s) every walk from s to j contains a path not longer, so $d^k(j) \geq \tilde{d}(j)$ for every k.
- Hence $d^{n-1} = d^n = \tilde{d}$.
- The labels never increase, and the algorithm stops at the first pass k with $d^k = d^{k-1}$, which is at most n; from that pass on the labels no longer change, so $d^k = d^{n-1} = \tilde{d}$.

□

Theorem 6: detection of a negative cycle

A negative cycle is reachable from s if and only if $d^n \neq d^{n-1}$. In that case, let $d^n(j) < d^{n-1}(j)$; reading the predecessors backwards from j, one row per pass, starting from row n of the table, gives a walk from s to j with n arcs that contains a negative cycle.

Proof (1/2)

If no negative cycle is reachable from s, then $d^n = d^{n-1}$ by the correctness of Bellman–Ford. Conversely, let $d^n(j) < d^{n-1}(j)$.

1. Every walk to j of length $d^n(j)$ with at most n arcs has exactly n arcs, otherwise $d^{n-1}(j) \leq d^n(j)$.

2. Let i be the predecessor of j in row n, so $d^n(j) = d^{n-1}(i) + d_{ij}$. A walk realizing $d^{n-1}(i)$ followed by (i, j) realizes $d^n(j)$, so it has exactly $n - 1$ arcs: $d^{n-1}(i) < d^{n-2}(i)$, and i has a predecessor in row $n - 1$.

Proof (2/2)

3. Repeating the argument down to row 1, whose predecessor is s, gives a walk W from s to j with n arcs and $d(W) = d^n(j)$.

4. W has $n+1$ nodes, so it repeats a node and contains a cycle C. Deleting C gives a walk to j with fewer than n arcs, of length

$$d(W) - d(C) \geq d^{n-1}(j) > d^n(j) = d(W).$$

Hence $d(C) < 0$, and C is reachable from s along W.

□

Proposition 5: the predecessor tree of Bellman–Ford

If no negative cycle is reachable from s, at the stop the arcs $(pred(j), j)$, for the nodes $j \neq s$ with finite label, form a shortest path tree of the reachable nodes.

Proof

Let $d = \tilde{d}$ be the final labels, and let $t(j)$ be the last pass in which the label of j decreased; $t(s) = 0$, since $d^k(s) = 0$ for all k. Let $p = pred(j)$.

1. In pass $t(j)$, $d(j) = d^{t(j)-1}(p) + d_{pj} \geq d(p) + d_{pj}$, since labels never increase.

2. By Bellman's equations, $d(j) \leq d(p) + d_{pj}$. Hence $d(j) = d(p) + d_{pj}$ and $d(p) = d^{t(j)-1}(p)$: the label of p did not change after pass $t(j) - 1$, so $t(p) < t(j)$.

3. Moving to the predecessor t strictly decreases, so the sequence ends in s, along tight arcs: the tree path from s to j has length $d(j)$.

□

Theorem 7: running time of Bellman–Ford

The Bellman–Ford algorithm runs in $O(n\,m)$ time.

Proof.

- A pass computes $d^k(j)$ for every j by scanning the backward star $\delta^-(j)$, so it scans every arc once: $O(n+m)$ time.
- There are at most n passes, and the total is $O(n(n+m))$, which is $O(n\,m)$ when $m \geq n-1$, as under (A1).

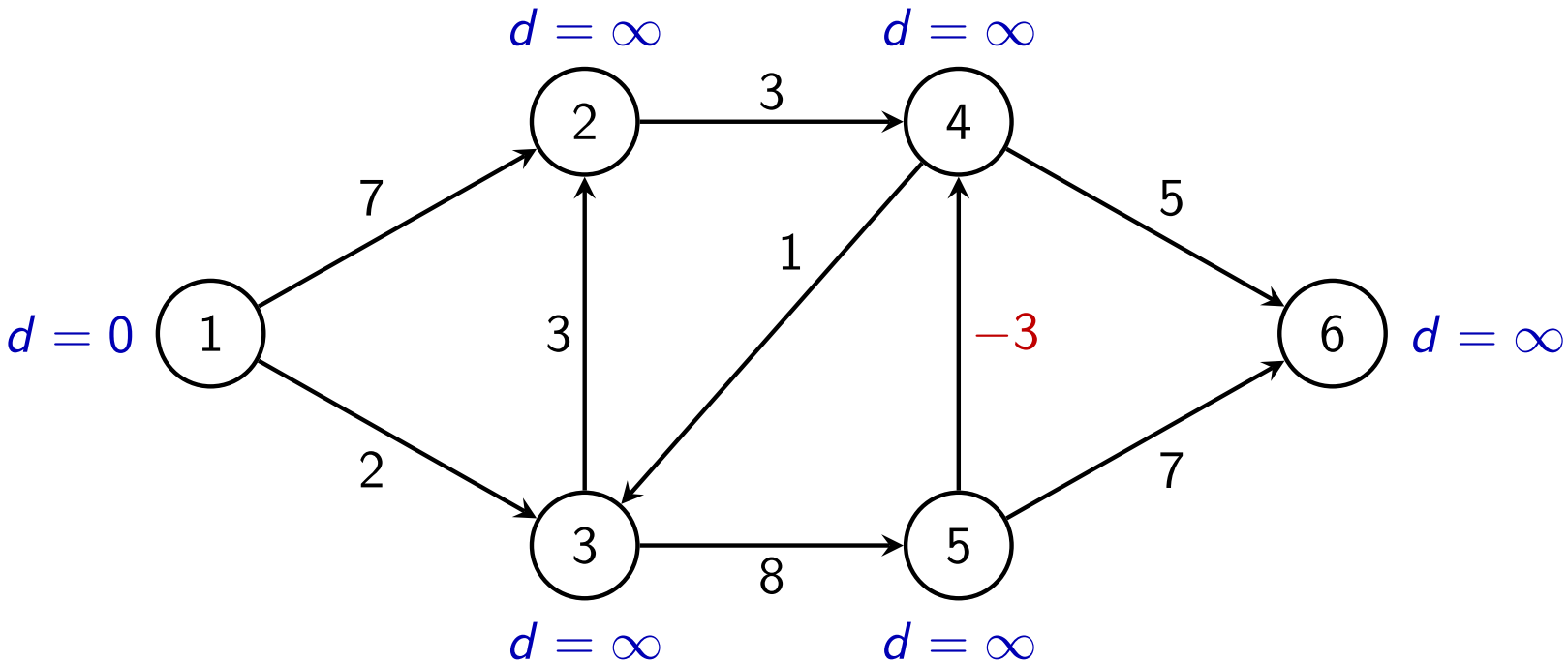
□

Example: Bellman–Ford with a negative arc

- We use the lengths d' of the first example with $d'_{54} = -3$ in place of 2.
- The two cycles have lengths 7 and $8 - 3 + 1 = 6$, so there is no negative cycle.
- In the next slides each pass computes the labels d^k from the labels d^{k-1} of the previous pass; a label that has just decreased is in red, and the arcs $(pred(j), j)$ are in blue.

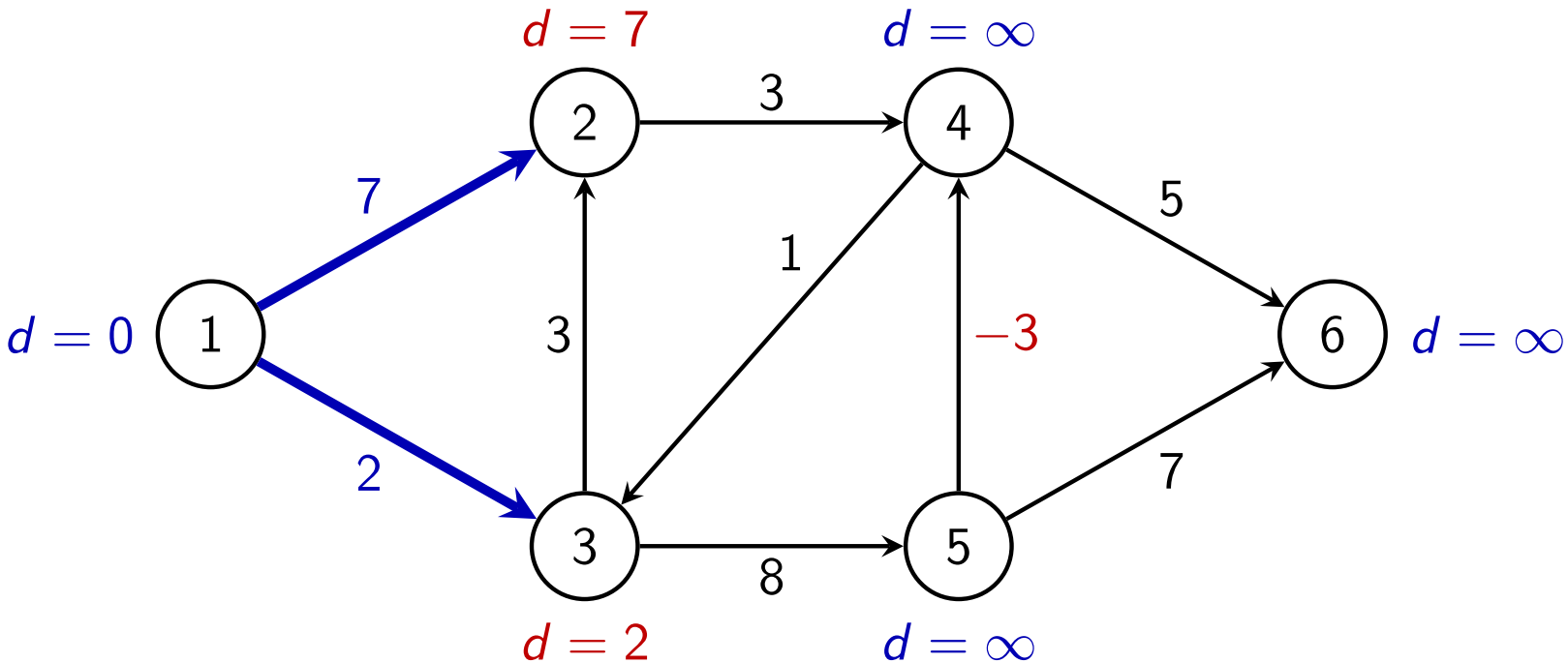
Example: pass 0

- $d^0 = (0, \infty, \infty, \infty, \infty, \infty)$.



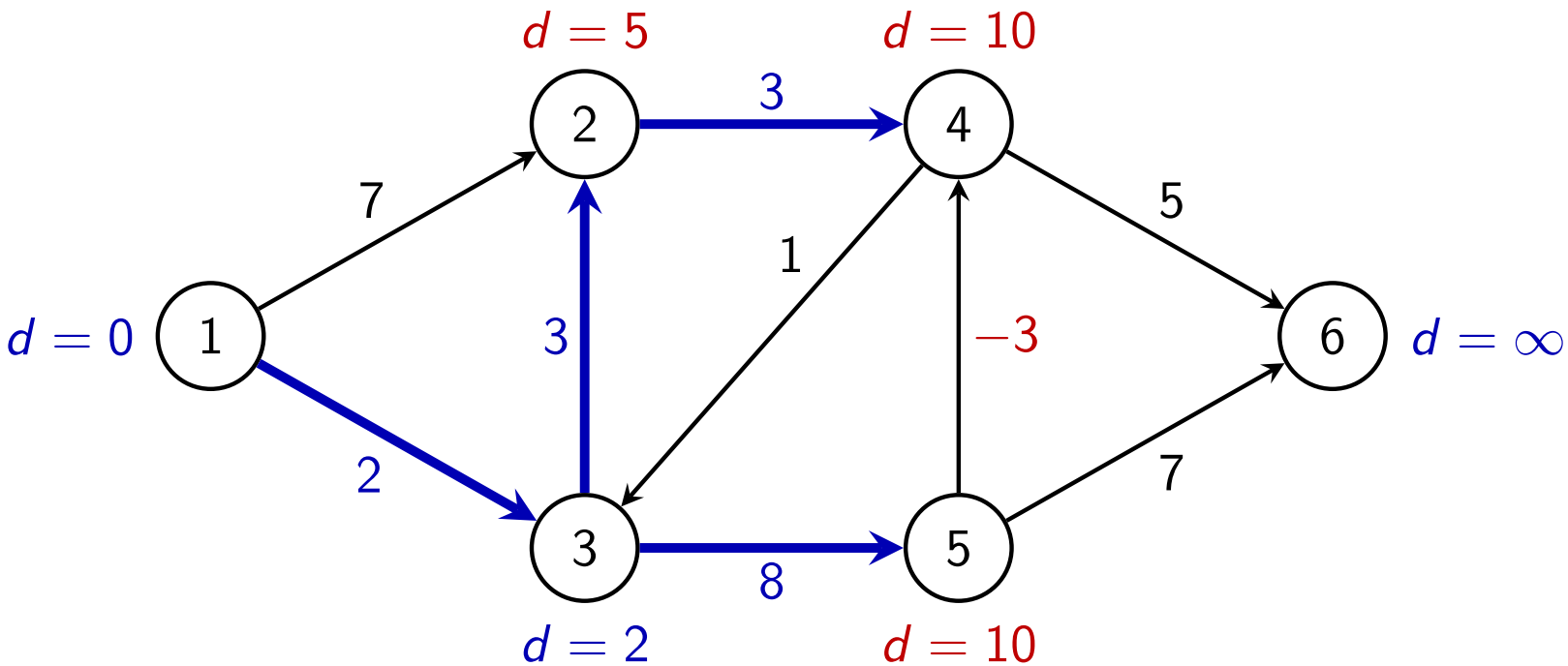
Example: pass 1

- $d^1(2) = \min\{\infty,\ 0+7\} = 7,\ pred(2) := 1;$
- $d^1(3) = \min\{\infty,\ 0+2\} = 2,\ pred(3) := 1.$



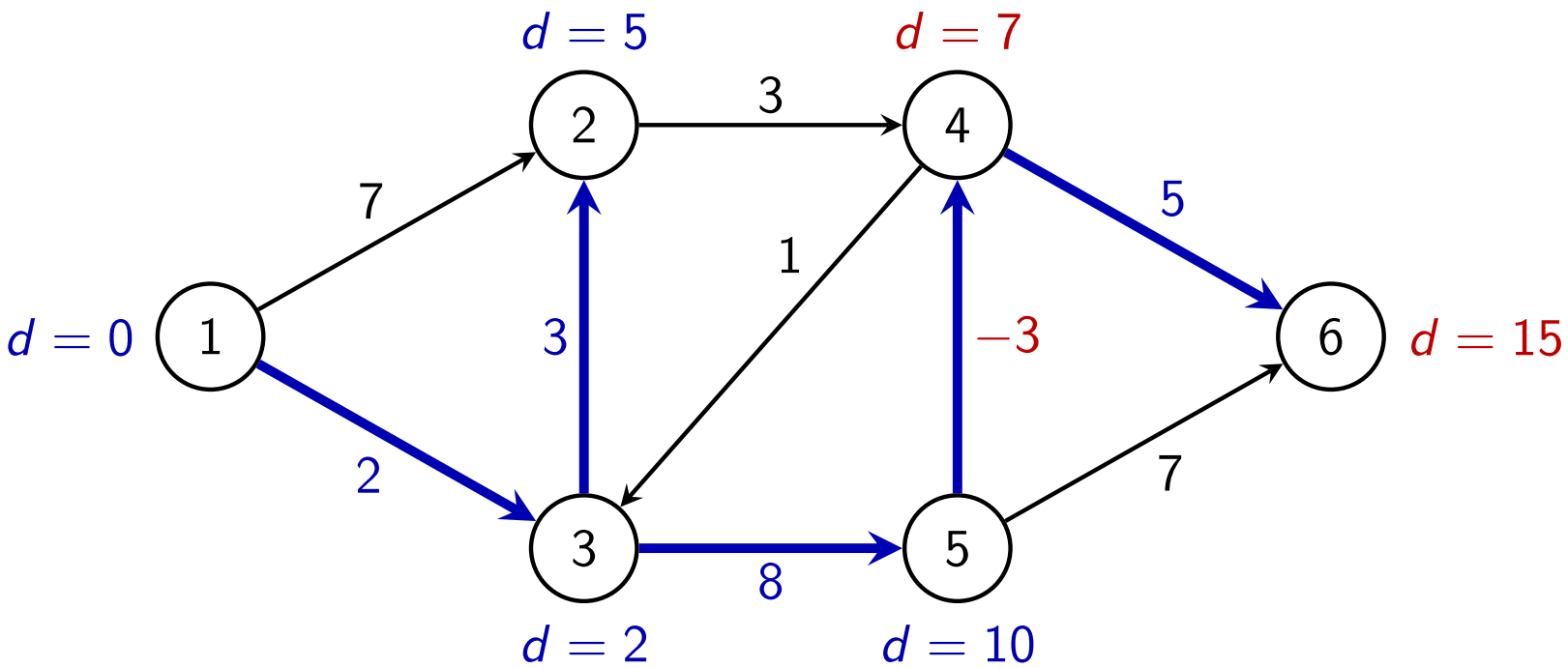
Example: pass 2

- $d^2(2) = \min\{7,\ 0+7,\ 2+3\} = 5$, $pred(2) := 3$;
- $d^2(4) = \min\{\infty,\ 7+3,\ \infty-3\} = 10$, $pred(4) := 2$;
- $d^2(5) = \min\{\infty,\ 2+8\} = 10$, $pred(5) := 3$.



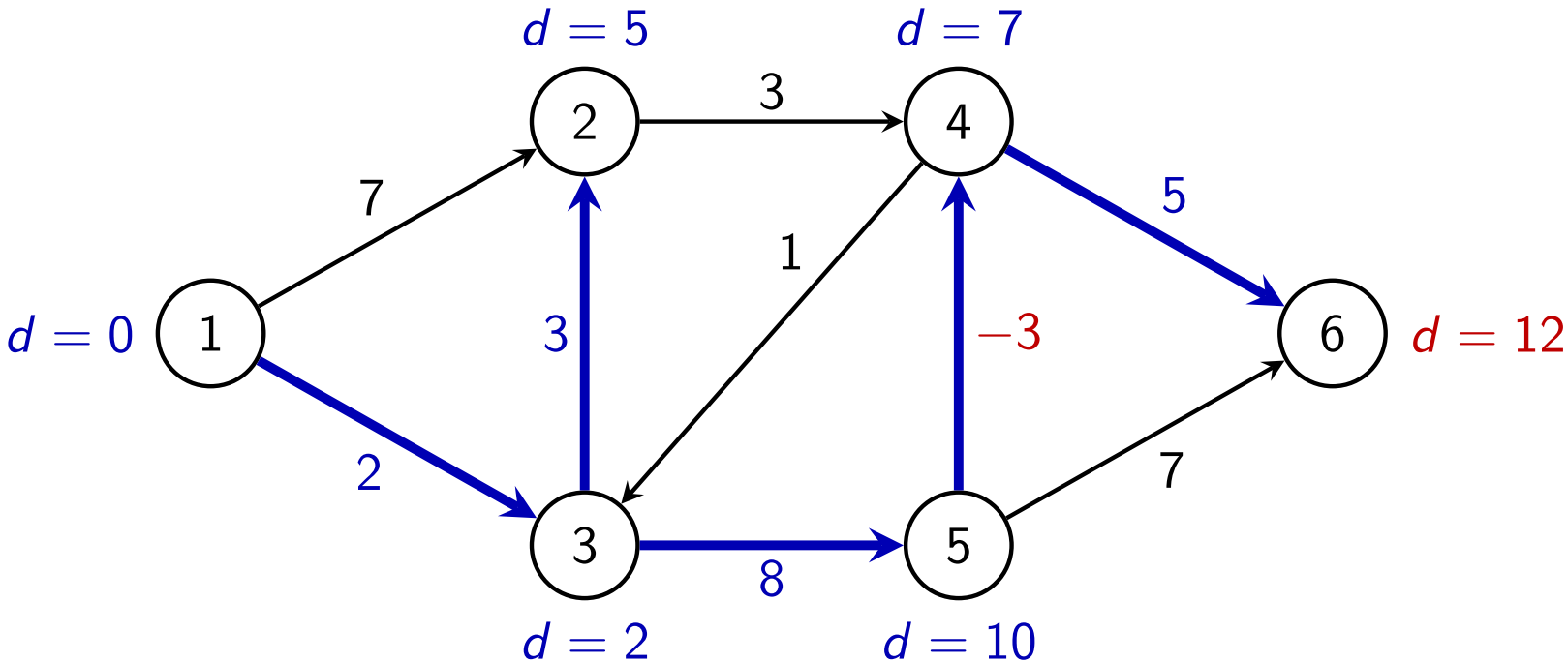
Example: pass 3

- $d^3(4) = \min\{10,\ d^2(2)+3,\ d^2(5)-3\} = \min\{10, 8, 7\} = 7$, with predecessor 5;
- $d^3(6) = \min\{\infty,\ 10+5,\ 10+7\} = 15$, $pred(6) := 4$.



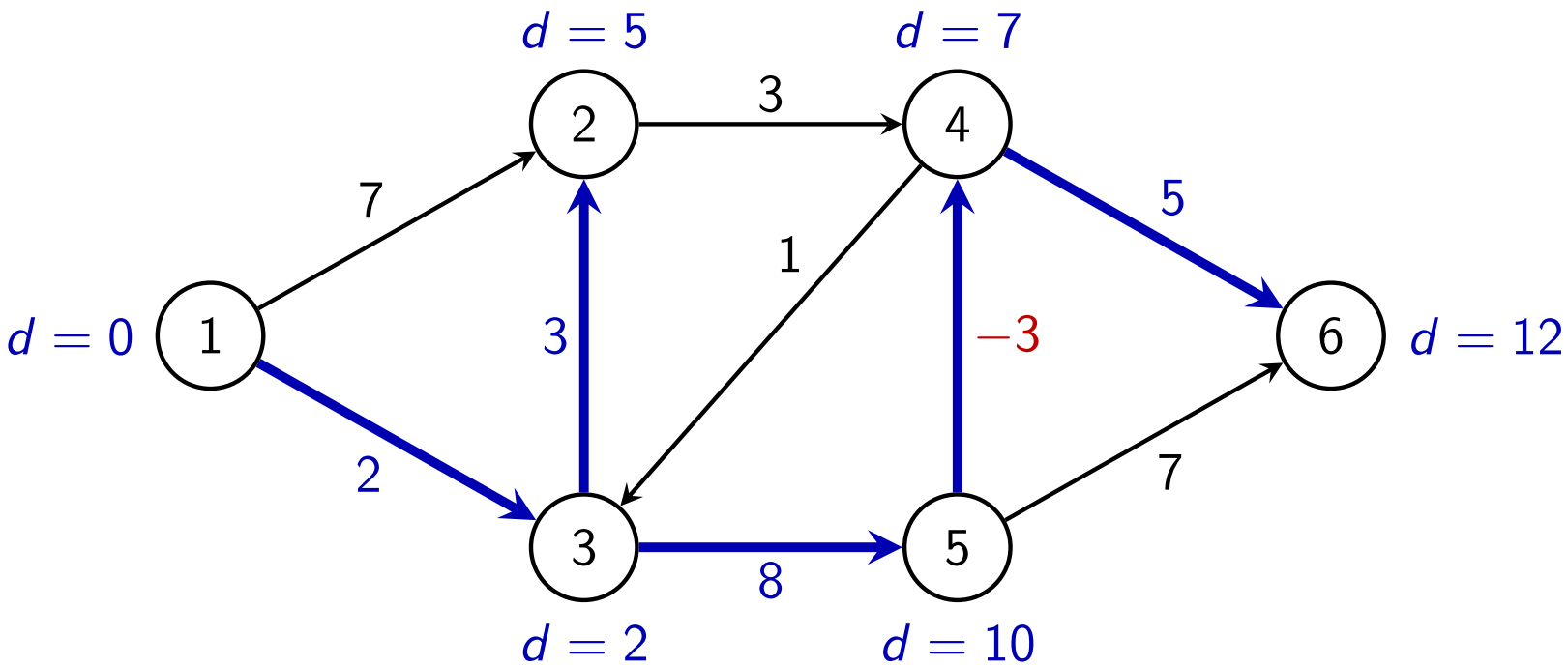
Example: pass 4

- $d^4(6) = \min\{15,\ 7+5,\ 10+7\} = 12$, $pred(6) = 4$ (unchanged); no other label decreases.



Example: pass 5

- No label decreases: $d^5 = d^4$, and the algorithm stops with $\tilde{d} = (0, 5, 2, 7, 10, 12)$, after 5 passes instead of the $n = 6$ allowed.



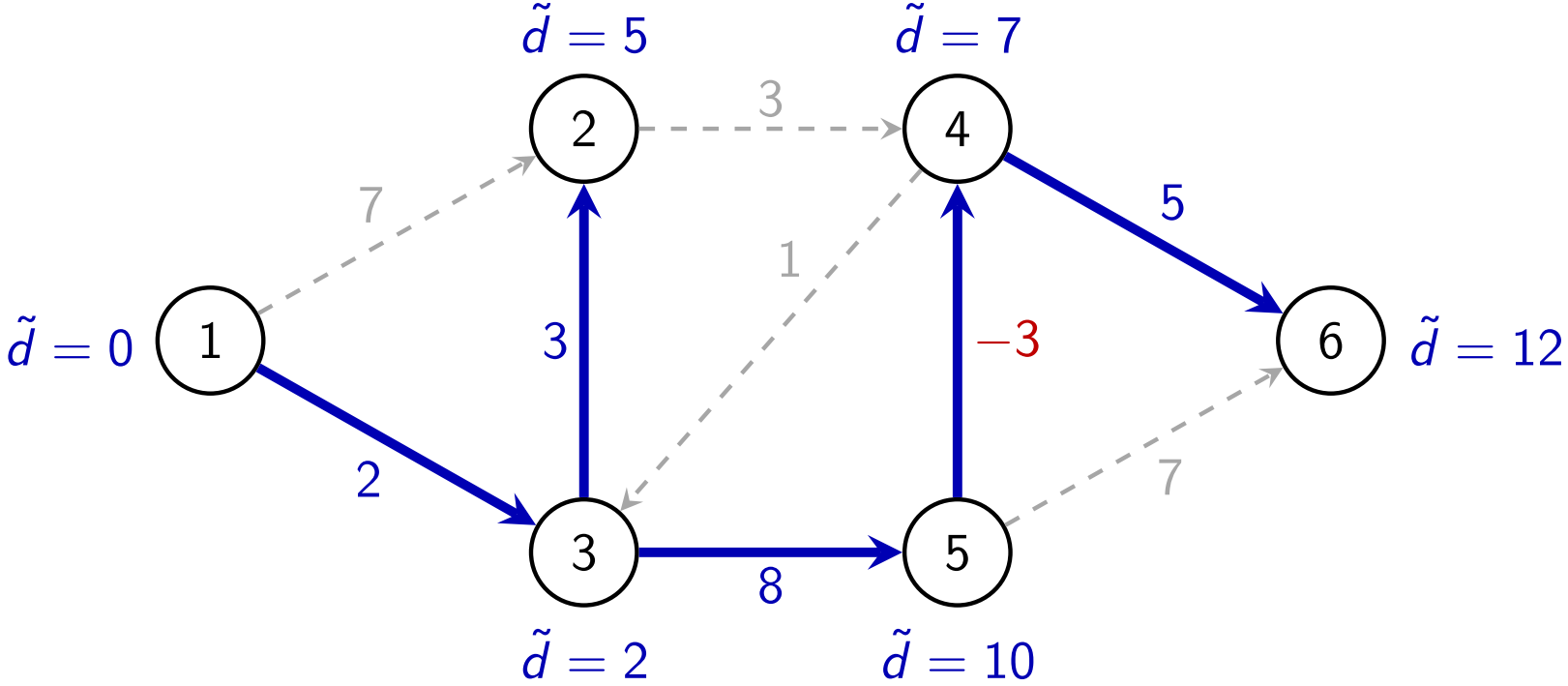
Example: the table of the passes

Each row is a pass k; next to a label that decreased, the new predecessor in brackets:

k	$d^k(1)$	$d^k(2)$	$d^k(3)$	$d^k(4)$	$d^k(5)$	$d^k(6)$
0	0	∞	∞	∞	∞	∞
1	0	7 (1)	2 (1)	∞	∞	∞
2	0	5 (3)	2	10 (2)	10 (3)	∞
3	0	5	2	7 (5)	10	15 (4)
4	0	5	2	7	10	12 (4)
5	0	5	2	7	10	12

Example: the result

The shortest path tree has the arcs $(1,3)$, $(3,2)$, $(3,5)$, $(5,4)$, $(4,6)$, and the shortest path to 6 is $1,3,5,4,6$, of length $2+8-3+5=12$.

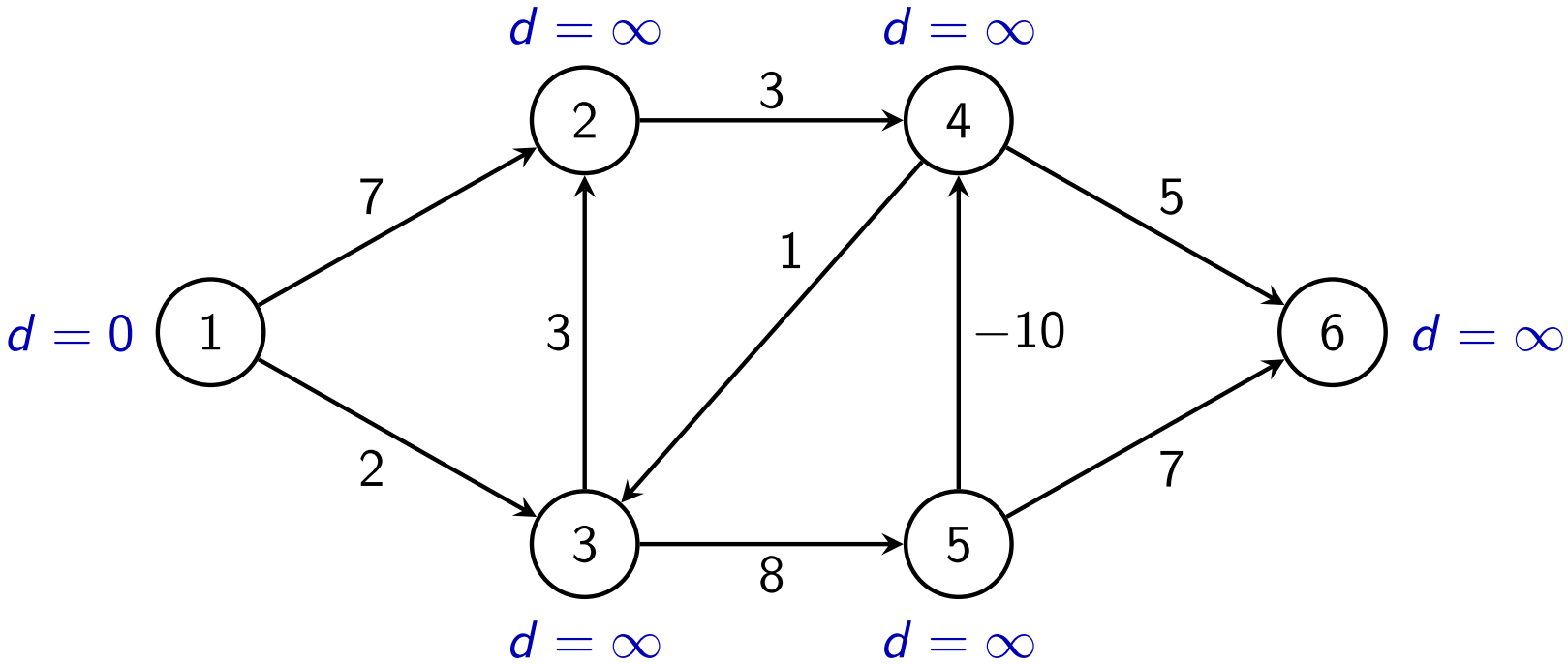


What happens when a negative cycle is reachable from the source?

We run the algorithm again, with $d''_{54} = -10$.

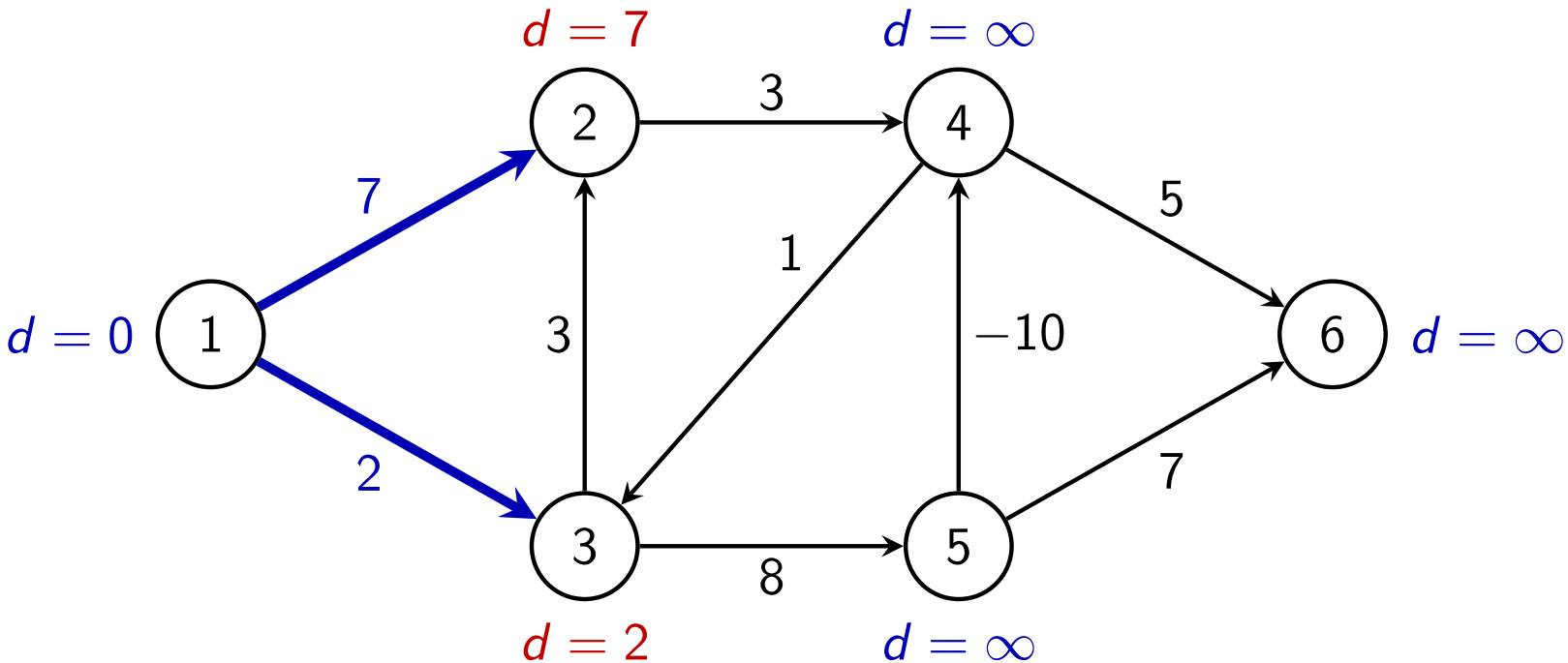
Example: Bellman–Ford detects a negative cycle

- With the lengths d'' of the negative cycle example ($d''_{54} = -10$) the algorithm runs all the $n = 6$ passes.
- Pass 0: $d^0 = (0, \infty, \infty, \infty, \infty, \infty)$.



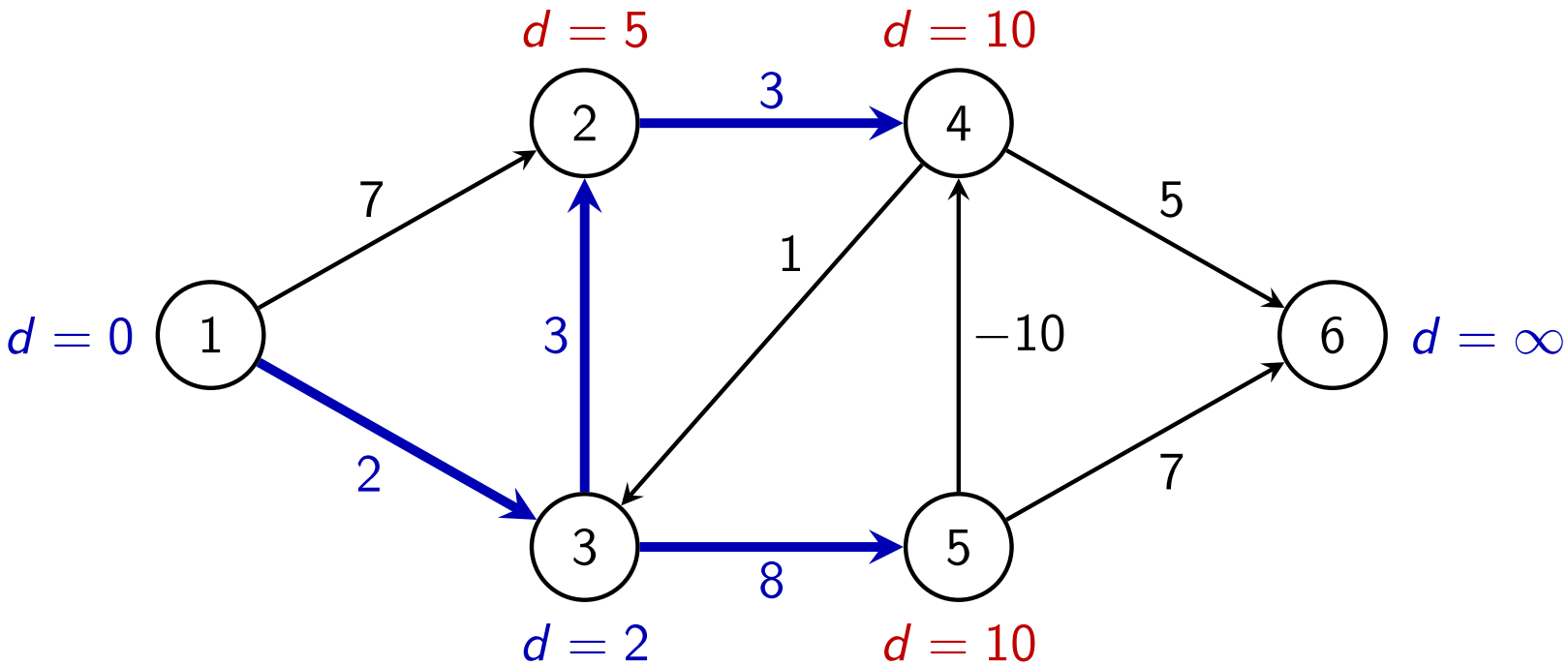
Example: pass 1

- $d^1(2) = \min\{\infty,\ 0+7\} = 7$, $pred(2) := 1$;
- $d^1(3) = \min\{\infty,\ 0+2\} = 2$, $pred(3) := 1$.



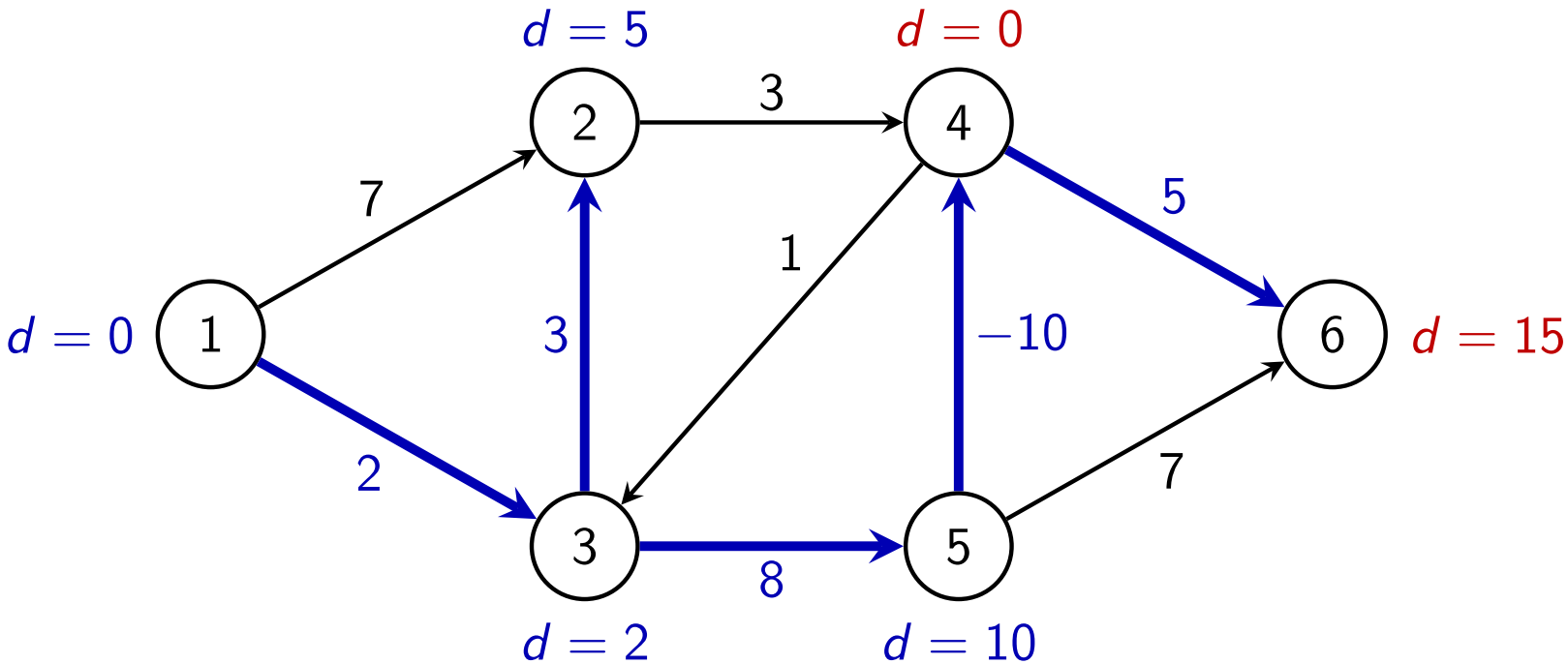
Example: pass 2

- $d^2(2) = \min\{7,\ 0+7,\ 2+3\} = 5$, $pred(2) := 3$;
- $d^2(4) = \min\{\infty,\ 7+3,\ \infty-10\} = 10$, $pred(4) := 2$;
- $d^2(5) = \min\{\infty,\ 2+8\} = 10$, $pred(5) := 3$.



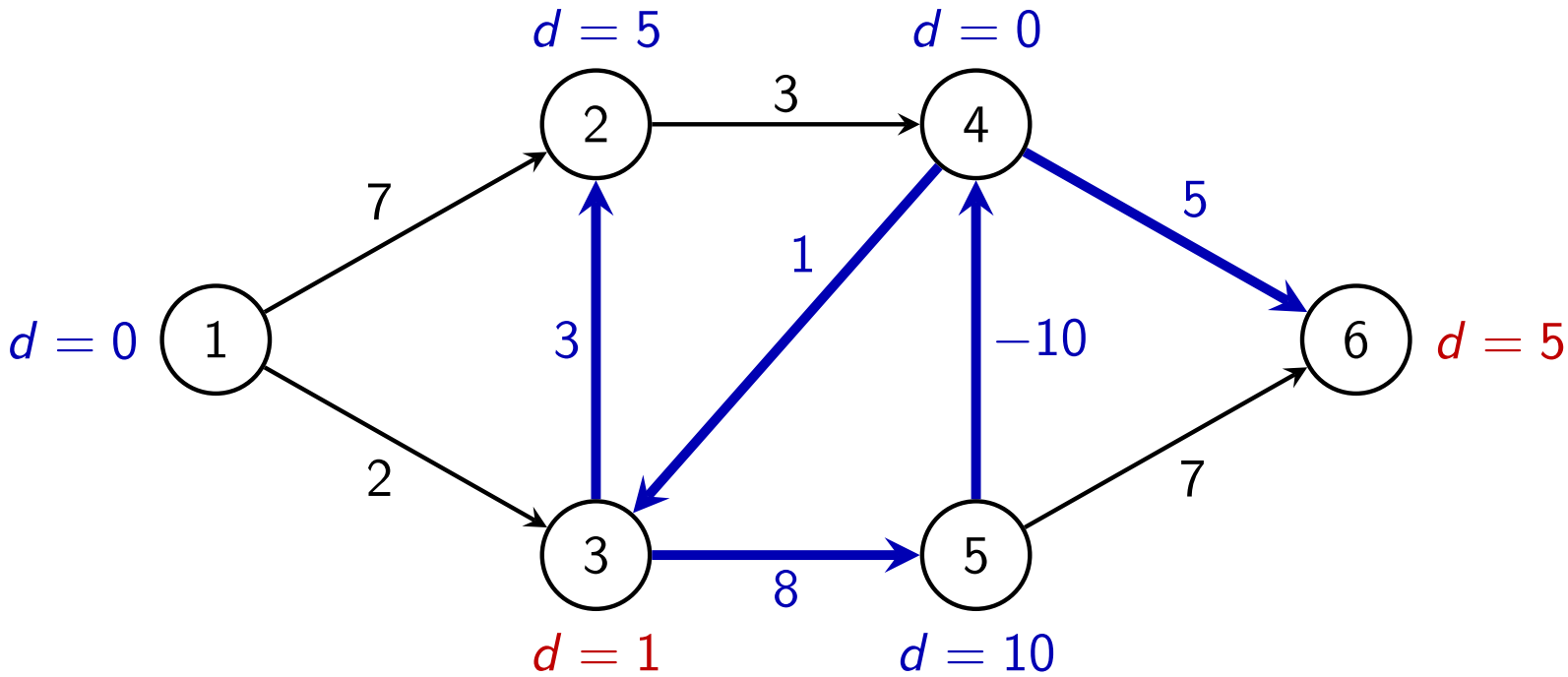
Example: pass 3

- $d^3(4) = \min\{10,\ 5+3,\ 10-10\} = 0$, $pred(4) := 5$;
- $d^3(6) = \min\{\infty,\ 10+5,\ 10+7\} = 15$, $pred(6) := 4$.



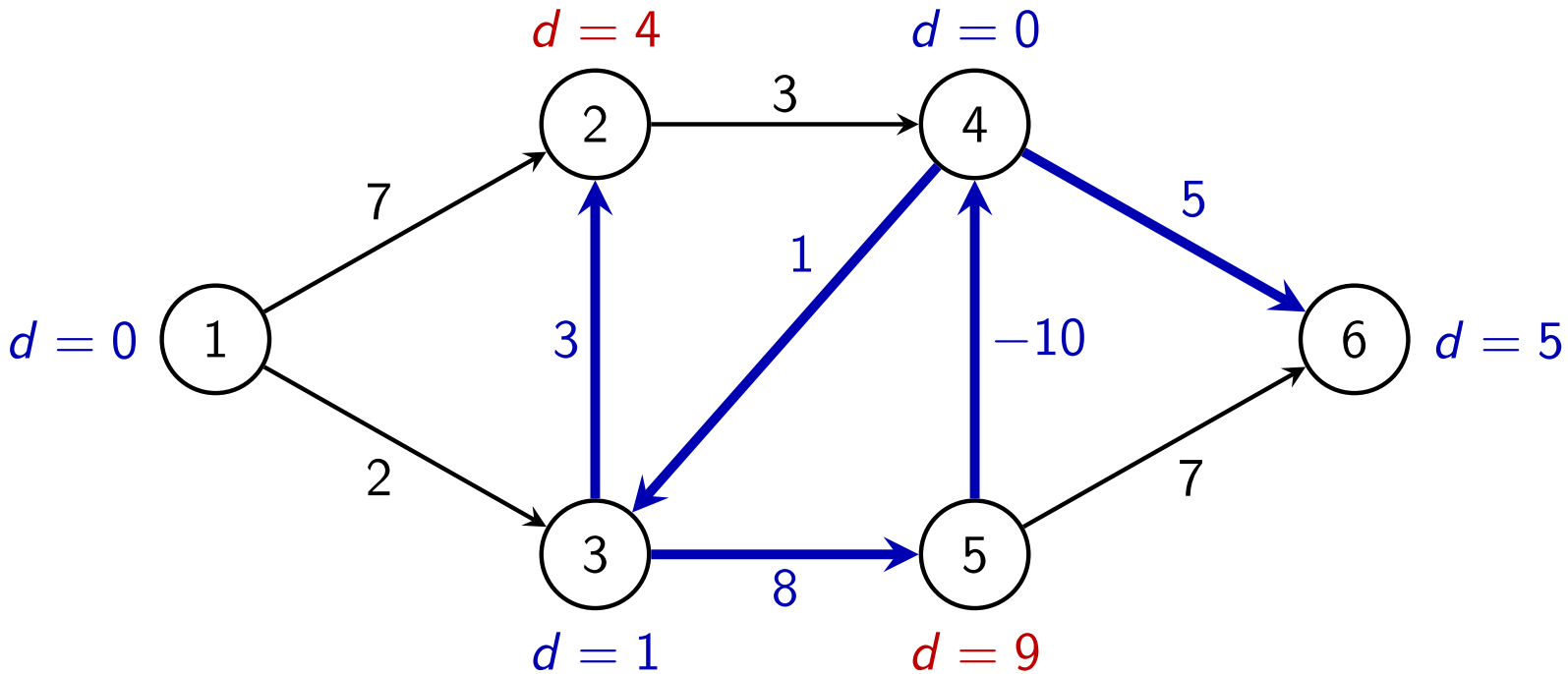
Example: pass 4

- $d^4(3) = \min\{2,\ 0+2,\ 0+1\} = 1$, $pred(3) := 4$;
- $d^4(6) = \min\{15,\ 0+5,\ 10+7\} = 5$, $pred(6) = 4$ (unchanged).



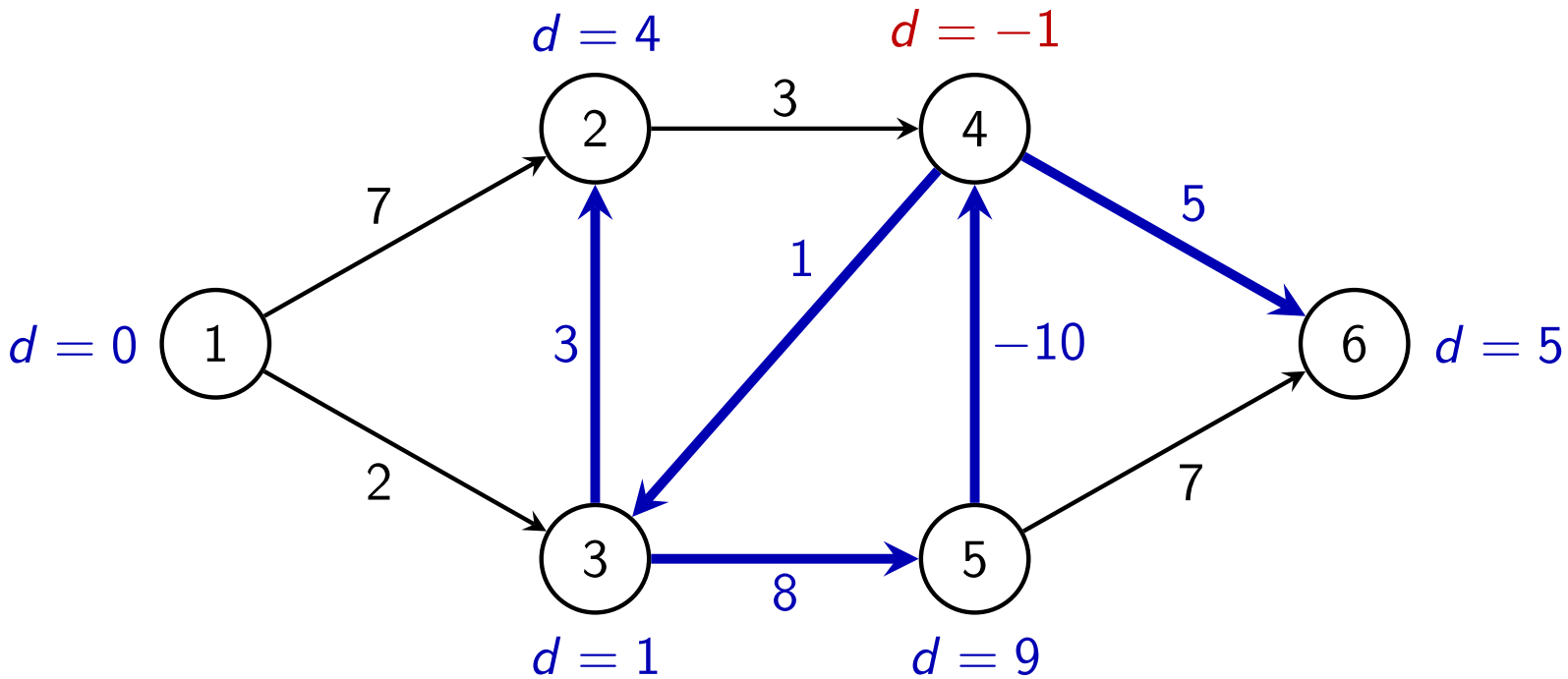
Example: pass 5

- $d^5(2) = \min\{5,\ 0+7,\ 1+3\} = 4$, $pred(2) = 3$ (unchanged);
- $d^5(5) = \min\{10,\ 1+8\} = 9$, $pred(5) = 3$ (unchanged).



Example: pass 6

- $d^6(4) = \min\{0,\ 4+3,\ 9-10\} = -1$, $pred(4) = 5$ (unchanged).
- The label of node 4 decreases in pass $n = 6$: $d^6 \neq d^5$, and a negative cycle is reachable from 1.



Example: the table of the passes

Next to a label that decreased, the new predecessor in brackets; in bold the entries read backwards:

k	$d^k(1)$	$d^k(2)$	$d^k(3)$	$d^k(4)$	$d^k(5)$	$d^k(6)$
0	0	∞	∞	∞	∞	∞
1	0	7 (1)	2 (1)	∞	∞	∞
2	0	5 (3)	2	10 (2)	10 (3)	∞
3	0	5	2	0 (5)	10	15 (4)
4	0	5	1 (4)	0	10	5 (4)
5	0	4 (3)	1	0	9 (3)	5
6	0	4	1	-1 (5)	9	5

Example: reading the predecessors backwards

- Reading backwards from node 4 (bold entries): row 6 gives $pred = 5$, row 5 for node 5 gives 3, row 4 for node 3 gives 4, then 5, 3, 1.
- The walk is

$$1,\ 3,\ 5,\ 4,\ 3,\ 5,\ 4,$$

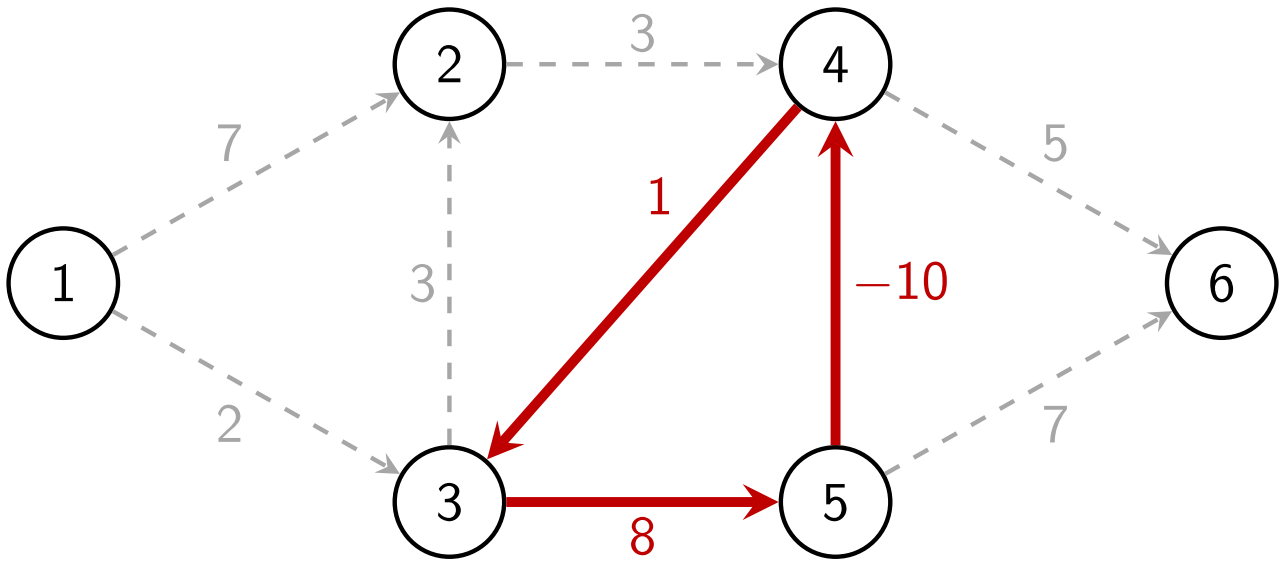
of length

$$2 + 8 - 10 + 1 + 8 - 10 = -1 = d^6(4).$$

- Node 3 repeats: the walk contains the cycle $3, 5, 4, 3$, of length -1.

Example: the negative cycle

The cycle $3, 5, 4, 3$ found by the algorithm, of length $8 - 10 + 1 = -1$ (red):



Summary

The labels d^k:

- $d^k(j)$ is the length of a shortest walk with at most k arcs, computed pass by pass with a recursion
- Each pass scans every arc once: $O(n\,m)$ time in total

Stop and detection:

- Without negative cycles reachable from s the algorithm stops within n passes with the distances
- A change in pass n reveals a negative cycle, found by reading the predecessors backwards

Dijkstra's algorithm

Introduction: Dijkstra's algorithm

- When all lengths are non-negative, a faster algorithm fixes one label at a time
- Dijkstra's algorithm is the method used in practice for road maps and networks: $O(n^2)$ time, or less with a heap
- We prove why it works, and we see why a single negative arc can break it

Permanent and temporary nodes

- In this section all lengths are non-negative: $d_{ij} \geq 0$ for every arc. Then every cycle has length ≥ 0, and (A2) holds.
- The nodes are **permanent**, when their label is final, or **temporary**.
- At every iteration the temporary node with the smallest label becomes permanent.

Algorithm: Dijkstra's algorithm

Input : a directed graph $G = (N, A)$ with lengths $d_{ij} \geq 0$, a source $s \in N$

Output: the distances $d(j) = \tilde{d}(j)$ and the predecessors $pred(j)$

1 **Step 0 (initialization).** Set $d(s) := 0$, $d(j) := +\infty$ for $j \neq s$; all nodes are temporary.

2 **Step 1 (selection).** If no temporary node has a finite label, `STOP`.

3 Otherwise make permanent the temporary node i with the smallest $d(i)$ (the smallest index, if more than one).

4 **Step 2 (update).** For each arc $(i, j) \in \delta^+(i)$ with j temporary: if $d(i) + d_{ij} < d(j)$, set $d(j) := d(i) + d_{ij}$ and $pred(j) := i$.

5 Go to Step 1.

Theorem 8: correctness of Dijkstra's algorithm

If $d_{ij} \geq 0$ for every arc, at every iteration of Dijkstra's algorithm:

(a) *every permanent node i has $d(i) = \tilde{d}(i)$;*

(b) *every temporary node j has as $d(j)$ the minimum length of a path from s to j whose nodes other than j are all permanent ($+\infty$ if there is none).*

At the stop, $d(j) = \tilde{d}(j)$ for every $j \in N$.

Proof (1/2)

Both properties hold after Step 0, with no permanent node. Let i be the node selected in Step 1.

1. $d(i) = \tilde{d}(i)$. Let P be any path from s to i, and let h be its first temporary node. The part of P up to h has length $\geq d(h)$ by (b), and the rest has length ≥ 0, since $d \geq 0$. Hence

$$d(P) \geq d(h) \geq d(i),$$

and by (b) the value $d(i)$ is the length of a path: $d(i) = \tilde{d}(i)$.

Proof (2/2)

2. *(b) after Step 2.* Let j be temporary and Q a path to j whose other nodes are permanent, with h the node before j. If $h = i$, then $d(Q) \geq \tilde{d}(i) + d_{ij} = d(i) + d_{ij}$, the value tested in Step 2.
3. If $h \neq i$, then h became permanent before i, through a path of permanent nodes of length $\tilde{d}(h)$; when h was selected, Step 2 gave $d(j) \leq \tilde{d}(h) + d_{hj} \leq d(Q)$.

Hence the new minimum is $\min\{d(j), d(i) + d_{ij}\}$, attained by a path, which is the label left by Step 2. At the stop, the temporary nodes have $d(j) = +\infty$: by (b) no arc leaves the permanent set towards them, so they are not reachable and $\tilde{d}(j) = +\infty$. □

Consequences

- For the s–t problem the algorithm can stop as soon as t is permanent, by (a).
- The arcs $(pred(j), j)$ form a shortest path tree: $pred(j)$ is permanent before j, and when it is last set $d(j) = d(pred(j)) + d_{pred(j)\,j}$ with both labels final.

Theorem 9: running time of Dijkstra's algorithm

With the labels in an array, Dijkstra's algorithm runs in $O(n^2)$ time.

Proof.

- There are at most n iterations, and Step 1 scans the temporary labels in $O(n)$ time.
- Step 2 scans the forward star of the selected node, and each node is selected once, so all the updates take $O(m)$ time.
- The total is $O(n^2 + m) = O(n^2)$, since $m \leq n(n-1)$.

□

With a heap

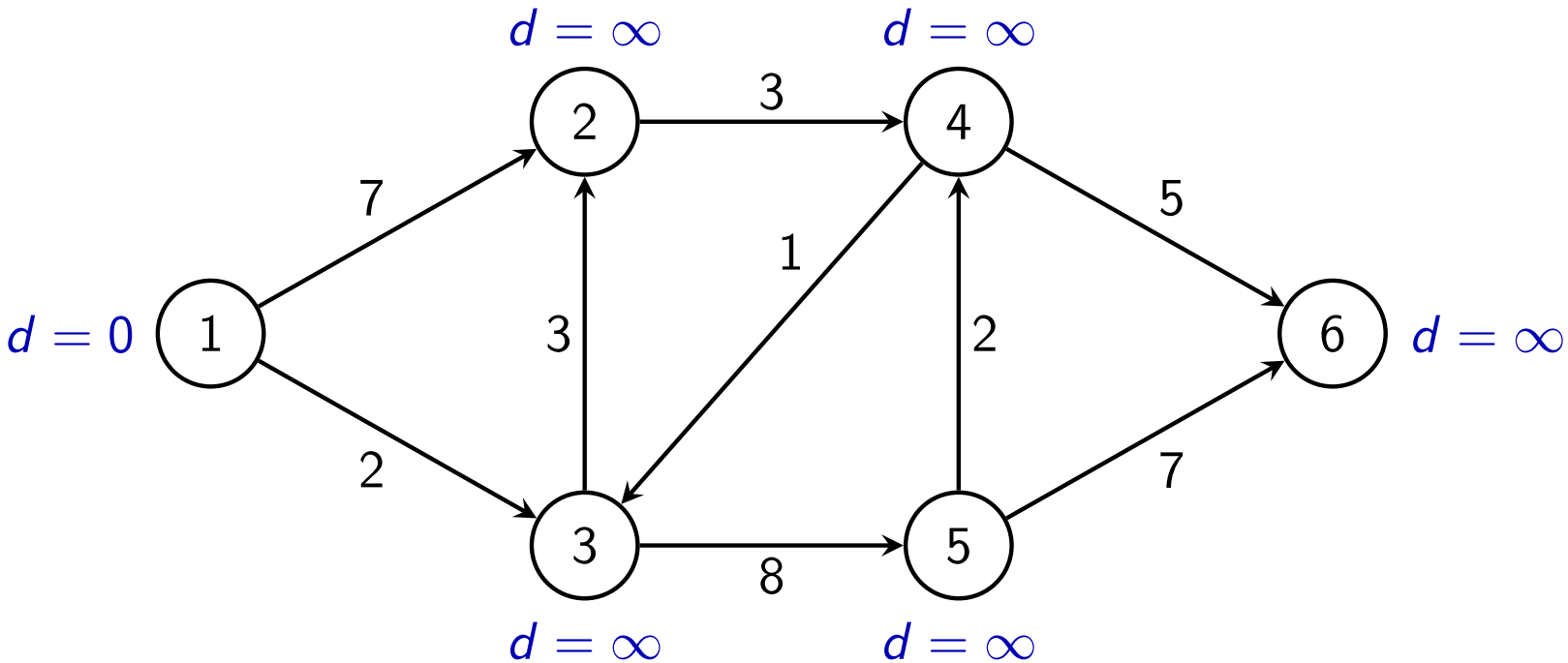
- With the temporary labels in a binary heap, Step 1 and each decrease of a label take $O(\log n)$ time.
- The total is
$$O((n+m)\log n),$$
better than $O(n^2)$ on sparse graphs.

Example: Dijkstra from node 1

- We run the algorithm with the lengths of the first example.
- In the next slides the selected node is in orange, the permanent nodes are shaded, a label that has just decreased is in red, and the arcs $(pred(j), j)$ are in blue.

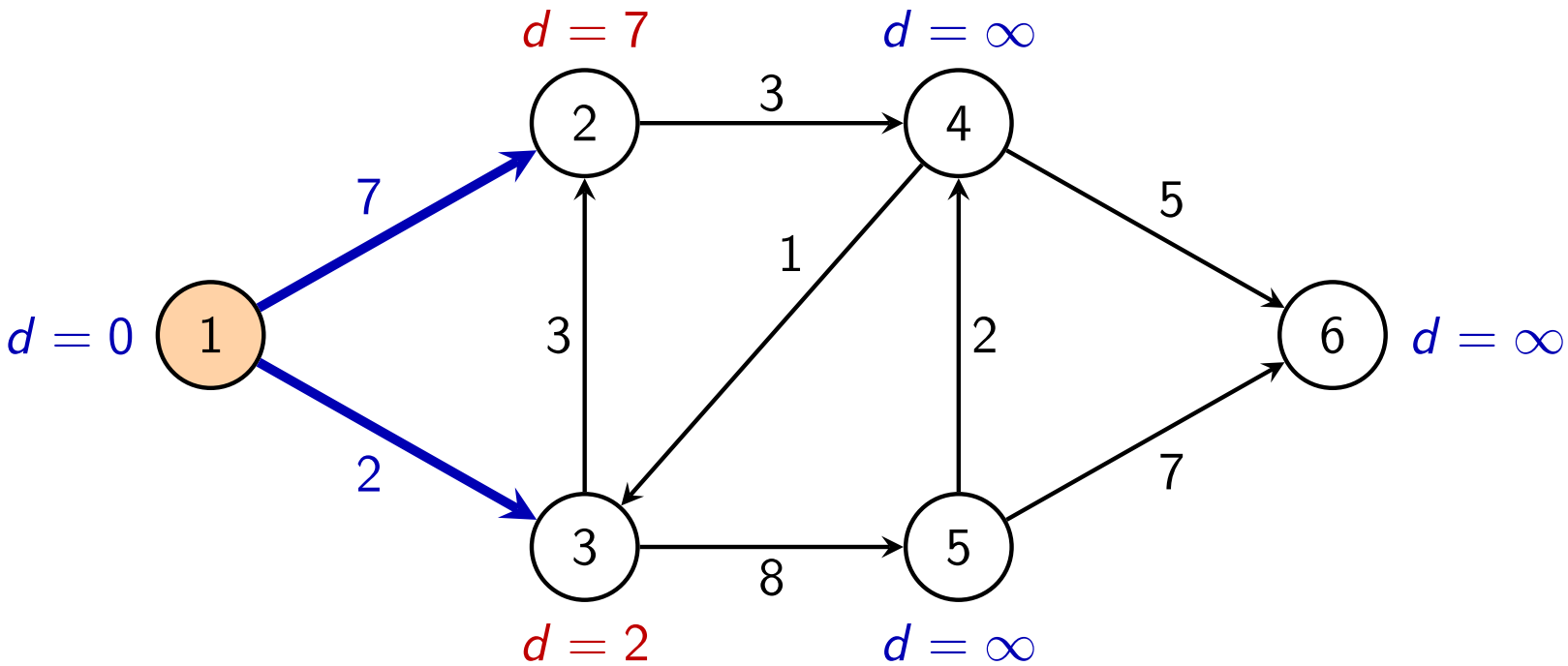
Example: Step 0

- $d(1) = 0$, $d(j) = +\infty$ for $j \neq 1$; all nodes are temporary.



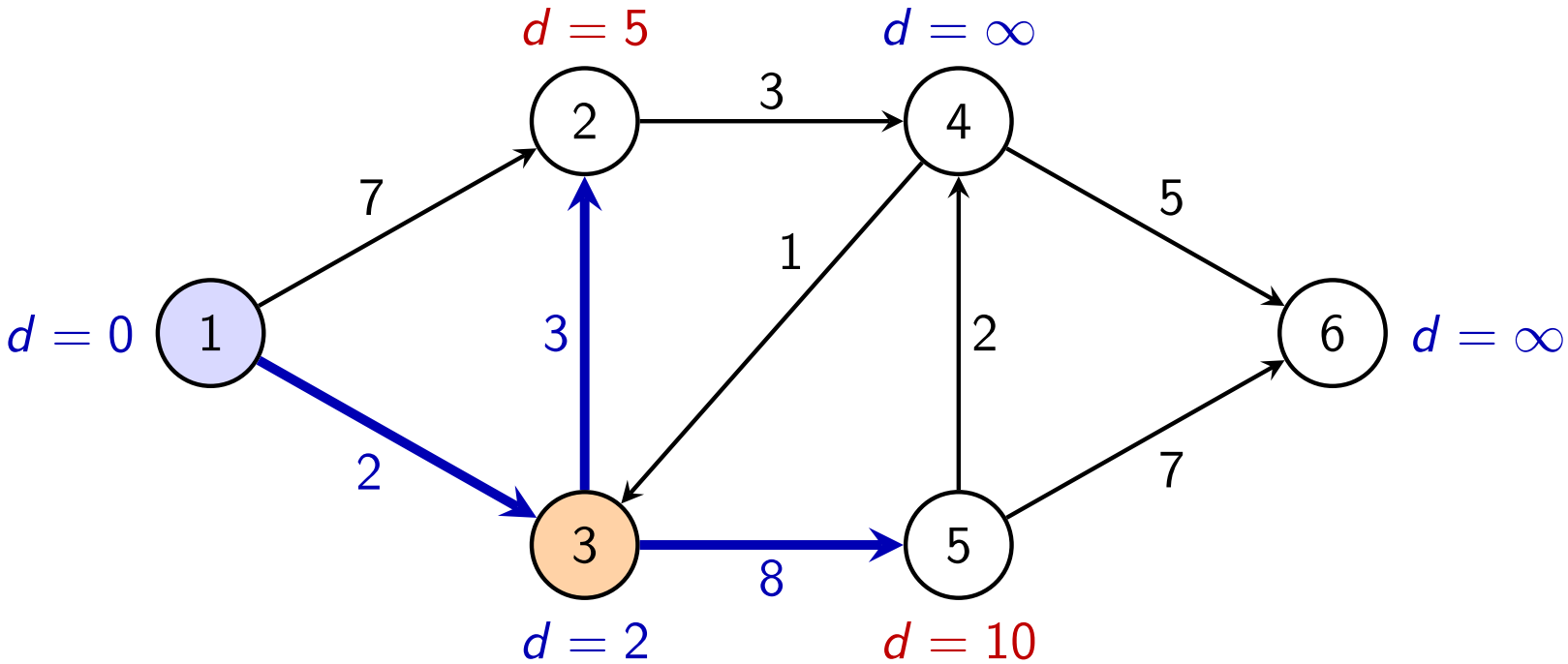
Example: iteration 1

- Node 1 ($d = 0$) becomes permanent.
- $(1, 2)$: $0 + 7 = 7 < \infty$, so $d(2) := 7$, $pred(2) := 1$;
- $(1, 3)$: $0 + 2 = 2 < \infty$, so $d(3) := 2$, $pred(3) := 1$.



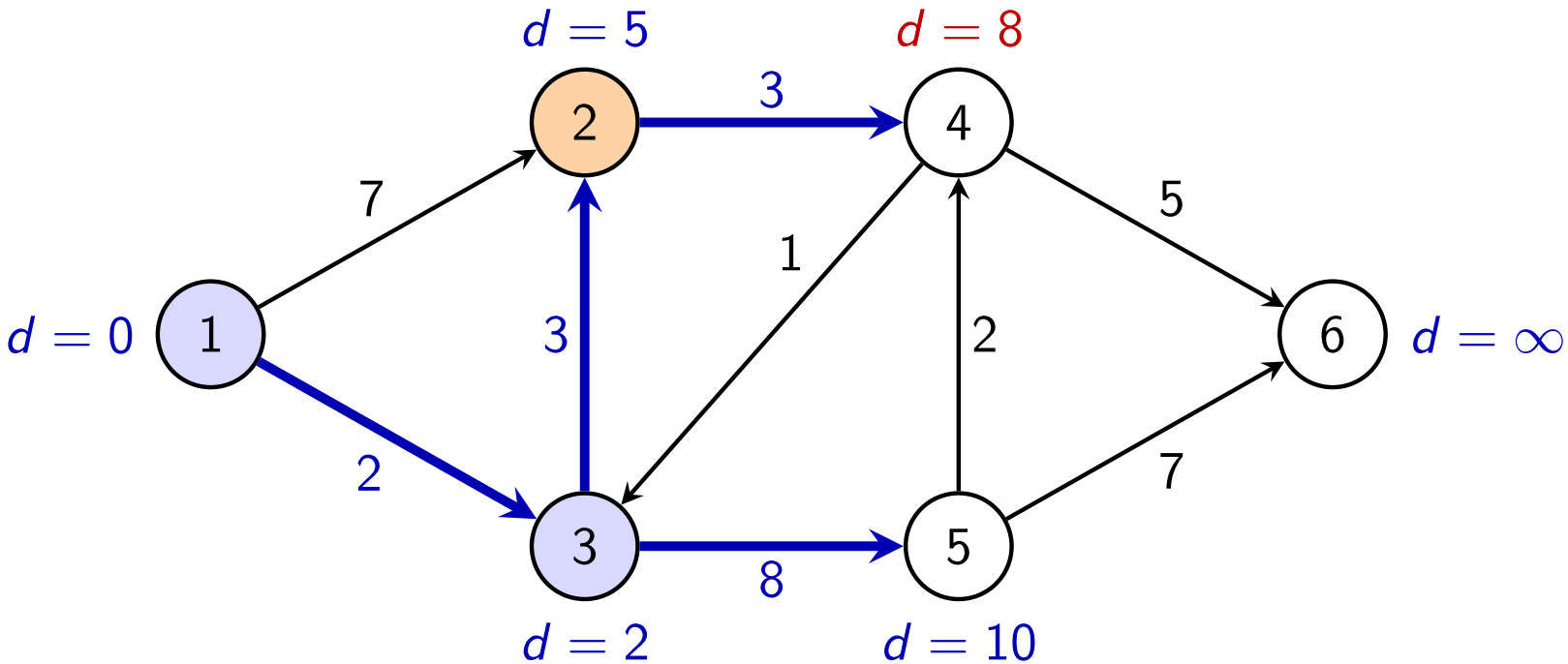
Example: iteration 2

- Node 3 ($d = 2$) becomes permanent.
- $(3, 2)$: $2 + 3 = 5 < 7$, so $d(2) := 5$, $pred(2) := 3$;
- $(3, 5)$: $2 + 8 = 10 < \infty$, so $d(5) := 10$, $pred(5) := 3$.



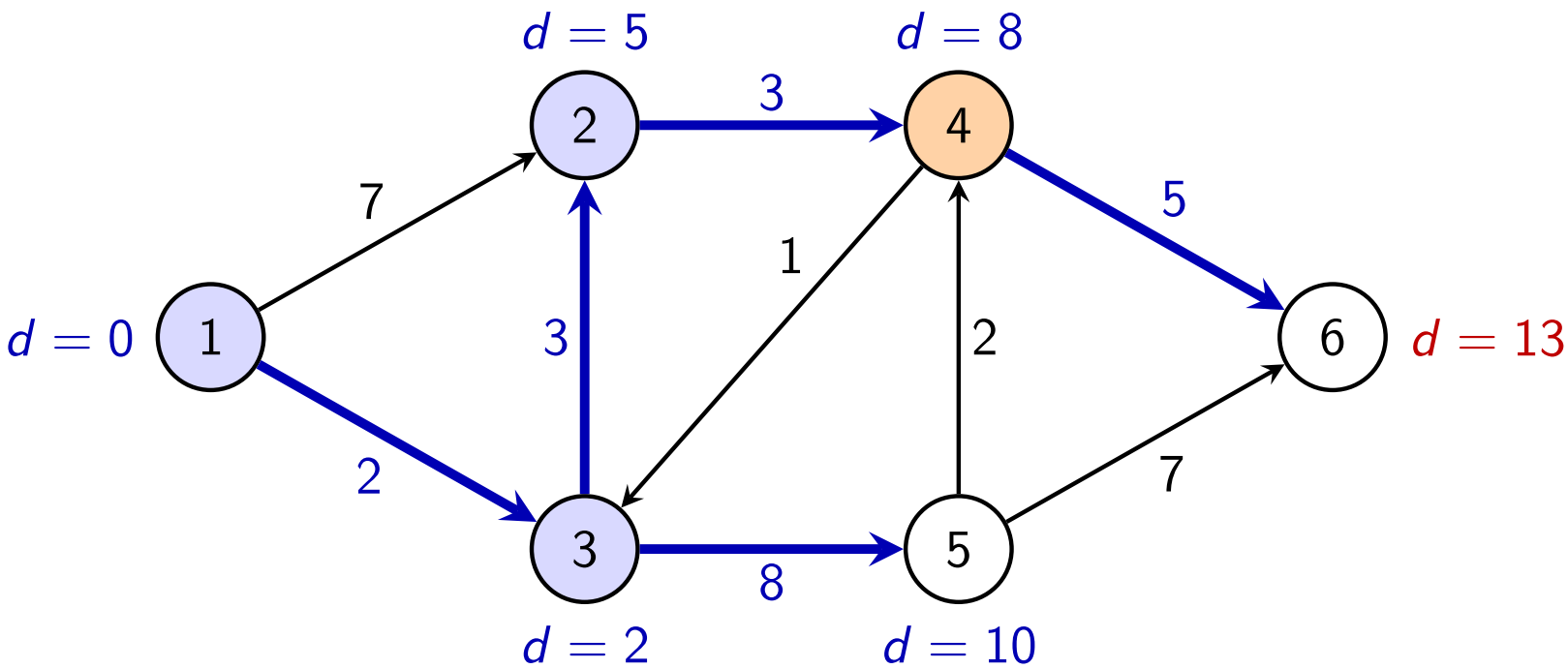
Example: iteration 3

- Node 2 ($d = 5$) becomes permanent.
- $(2, 4)$: $5 + 3 = 8 < \infty$, so $d(4) := 8$, $pred(4) := 2$.



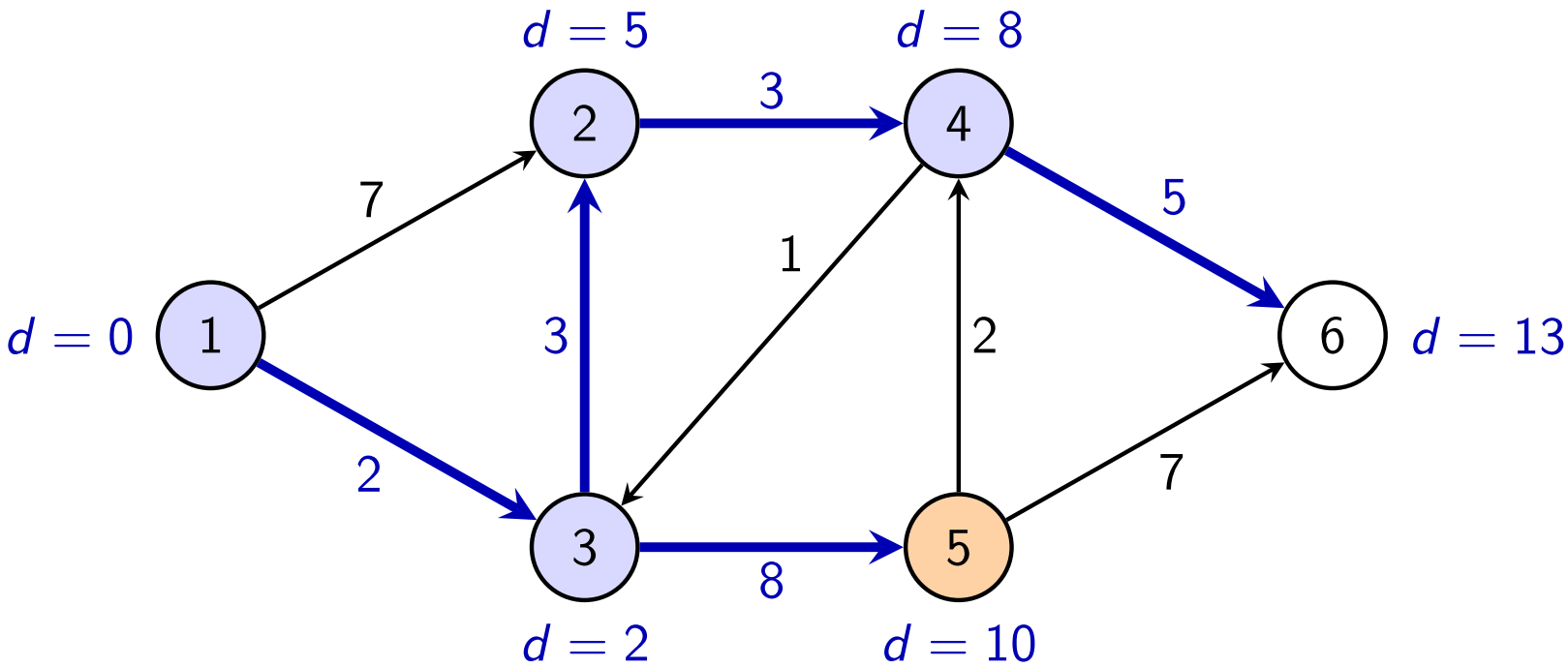
Example: iteration 4

- Node 4 ($d = 8 < 10$) becomes permanent; $(4, 3)$ ends in a permanent node.
- $(4, 6)$: $8 + 5 = 13 < \infty$, so $d(6) := 13$, $pred(6) := 4$.



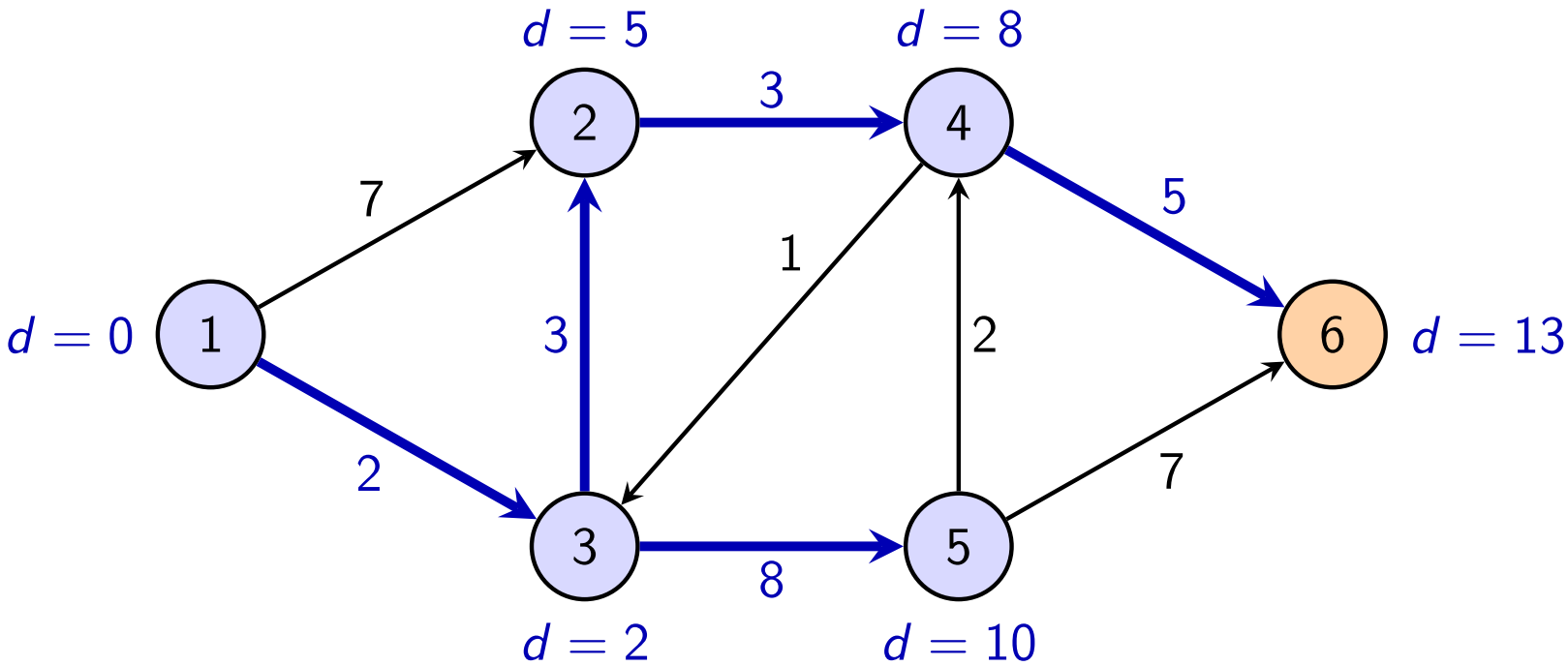
Example: iteration 5

- Node 5 ($d = 10$) becomes permanent. The arc $(5, 6)$ offers $10 + 7 = 17 > 13$, and the arc $(5, 4)$ ends in a permanent node: no change.



Example: iteration 6

- Node 6 ($d = 13$) becomes permanent; it has no leaving arc. No temporary node is left: `STOP`.



Example: the table of the iterations

Each row is an iteration: the selected node becomes permanent (bold label), and a label that decreased has the new predecessor in brackets.

iteration	selected i	$d(1)$	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$
1	1	0	7 (1)	2 (1)	∞	∞	∞
2	3	0	5 (3)	2	∞	10 (3)	∞
3	2	0	5	2	8 (2)	10	∞
4	4	0	5	2	8	10	13 (4)
5	5	0	5	2	8	10	13
6	6	0	5	2	8	10	13

Example: the result

- In iteration 2 the label of node 2 drops from 7 to $2 + 3 = 5$; in iteration 5 the arc $(5, 6)$ offers $10 + 7 = 17 > 13$, and the arc $(5, 4)$ ends in a permanent node.

- The nodes become permanent in the order $1, 3, 2, 4, 5, 6$, with

$$\tilde{d} = (0, 5, 2, 8, 10, 13);$$

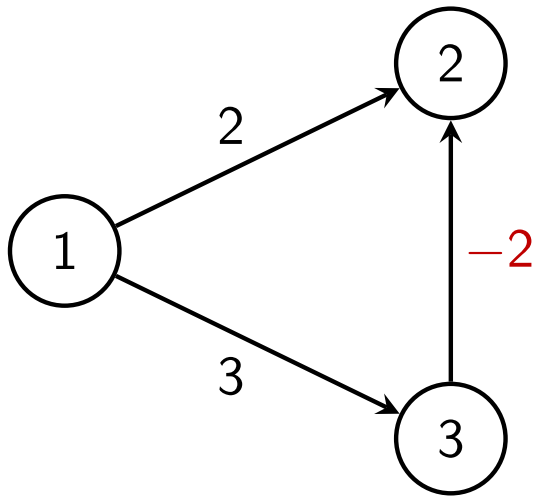
the predecessors give the shortest path tree found with Bellman's equations.

Why do we need $d_{ij} \geq 0$?

A single negative arc is enough to break Dijkstra's algorithm.

Example (1/3)

In the graph below, with $s = 1$, the algorithm makes node 2 permanent with $d(2) = 2$, then node 3 with $d(3) = 3$; the arc $(3, 2)$ ends in a permanent node and is ignored. The path $1, 3, 2$ has length $3 - 2 = 1 < 2$.



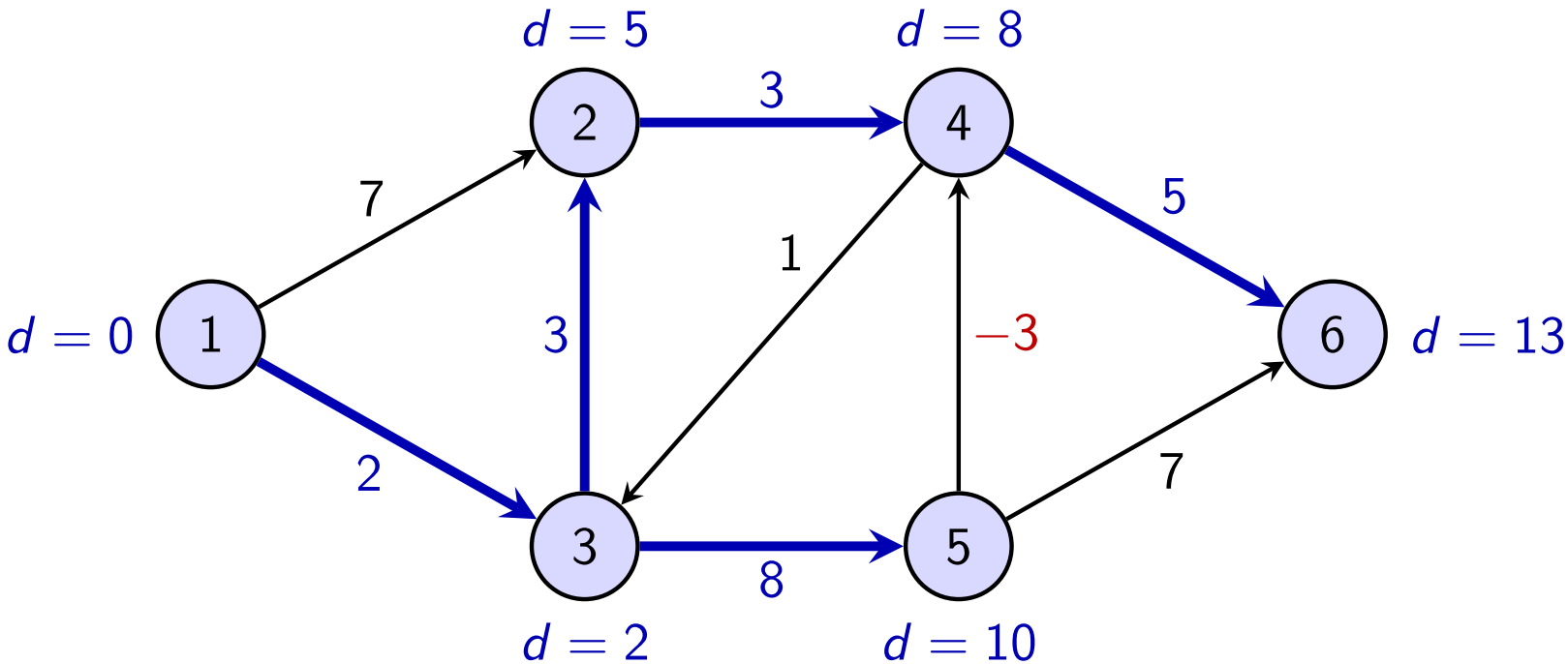
Example (2/3)

- On the reference graph with the lengths d' of the Bellman–Ford example ($d'_{54} = -3$), Dijkstra's algorithm makes the nodes permanent in the order $1, 3, 2, 4, 5, 6$ and returns $d(4) = 8$ and $d(6) = 13$.

- Node 4 is permanent before node 5, so the shorter path $1, 3, 5, 4$ of length 7 is missed, and with it the path to 6 of length 12.

Example (3/3)

The labels returned by Dijkstra's algorithm with $d'_{54} = -3$: when node 5 becomes permanent, the arc $(5, 4)$ ends in a permanent node and is ignored.



Where the proof fails

- The proof of the correctness of Dijkstra's algorithm uses $d \geq 0$ in one point: the rest of P after its first temporary node has length ≥ 0.
- A negative arc breaks exactly this step.
- With a negative arc we use the Bellman–Ford algorithm instead, or the topological order if the graph is acyclic.

Summary

Dijkstra's algorithm:

- With non-negative lengths it makes permanent, at every iteration, the temporary node with the smallest label, whose label is then final
- It runs in $O(n^2)$ time, $O((n+m)\log n)$ with a heap

Negative lengths:

- With a negative arc it can fail
- The Bellman–Ford algorithm is used instead

Typical mistakes

Introduction: Typical mistakes

- We collect the mistakes that are made most often with shortest paths
- They concern the choice of the algorithm, the sign of the potentials and the stopping tests
- Check them before running an algorithm by hand

Typical mistakes (1/4)

Observation 2: Running Dijkstra's algorithm with a negative arc

Its labels can be wrong, as with $d'_{54} = -3$ on the reference graph; use Bellman–Ford, or the topological order if the graph is acyclic.

Observation 3: Adding a constant to every length

It makes the lengths non-negative but changes the shortest paths: a path with k arcs grows by k times the constant.

Typical mistakes (2/4)

Observation 4: Swapping the sign of the potentials

With $+1$ in the tail of $\boldsymbol{B}$ the dual constraints are $\pi_i - \pi_j \leq d_{ij}$ and the optimal potentials are $\pi_i = -\tilde{d}(i)$.

Observation 5: Skipping the test of pass n

After $n-1$ passes the labels are the distances only without negative cycles; a change in pass n reveals one.

Typical mistakes (3/4)

Observation 6: Expecting Bellman–Ford to see every negative cycle

It detects only the cycles reachable from s, while a negative cycle anywhere in G makes the LP model unbounded.

Observation 7: Updating a permanent node in Dijkstra's algorithm

Its label is final; only temporary nodes are updated.

Typical mistakes (4/4)

Observation 8: Looking for a topological order in a graph with cycles

It exists only in acyclic graphs: the reference graph has none.

Summary

Choosing the algorithm:

- Non-negative lengths: Dijkstra; acyclic graph: topological order; otherwise Bellman–Ford
- Never shift the lengths by a constant

Running it:

- Bellman–Ford: always do the test of pass n; it sees only the cycles reachable from s
- Dijkstra: permanent labels are never updated; potentials: $\pi_i = -\tilde{d}(i)$

In summary

Introduction: In summary

- We collect the key ideas of the chapter on shortest paths
- The last slide gathers the results on the reference graph, for every set of lengths used in the chapter

Summary (1/3)

- The distance $\tilde{d}(j)$ is the length of a shortest path from s to j; it is well defined when no negative cycle is reachable from s (A2).
- Subpaths of shortest paths are shortest; the distances satisfy Bellman's equations, and a shortest path tree exists.
- Feasible potentials, $\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j \geq 0$, and a path of tight arcs certify optimality.

Summary (2/3)

- The LP model is a unit flow, $\boldsymbol{B}\boldsymbol{x} = \boldsymbol{b}$, $\boldsymbol{x} \geq 0$; its dual is

$$\max\{\pi_s - \pi_t : \pi_i - \pi_j \leq d_{ij}\},$$

  optimal at $\pi_i = -\tilde{d}(i)$.

- Acyclic graphs: topological order, $O(n + m)$, any lengths.
- Bellman–Ford: $O(n\,m)$, any lengths, detects negative cycles.
- Dijkstra: $O(n^2)$, non-negative lengths.

Summary (3/3)

The reference graph at a glance.

lengths d (Dijkstra)	$\tilde{d} = (0, 5, 2, 8, 10, 13)$, order $1, 3, 2, 4, 5, 6$
LP models	$z(\mathrm{SP}(1, 6)) = 13$, $z(\mathrm{SP}(1)) = 38$
lengths d', $d'_{54} = -3$ (Bellman–Ford)	$\tilde{d} = (0, 5, 2, 7, 10, 12)$, 5 passes
lengths d'', $d''_{54} = -10$	negative cycle $3, 5, 4, 3$ of length -1
acyclic graph, arc $(3, 4)$	order $1, 3, 2, 5, 4, 6$; $\tilde{d} = (0, 5, 2, 3, 10, 8)$